

EE 186 – Introduction to Embedded Systems

Lecture 5

Zerina Kapetanovic

Today we are talking about...

- Timers
 - Background
 - Timer Anatomy
 - Capture and Compare
 - Pulse Width Modulation
- SysTick Timers
- Virtual Timers



What is a timer?

A **timer** is a peripheral for tracking time

One of the most important features of a microcontroller!

Many use cases:

Scheduled Tasks – trigger an interrupt to read in sensor data, update a display, transmit a data packet, etc.

Generate PWM Signals – control the speed of a motor, the brightness of an LED, etc.

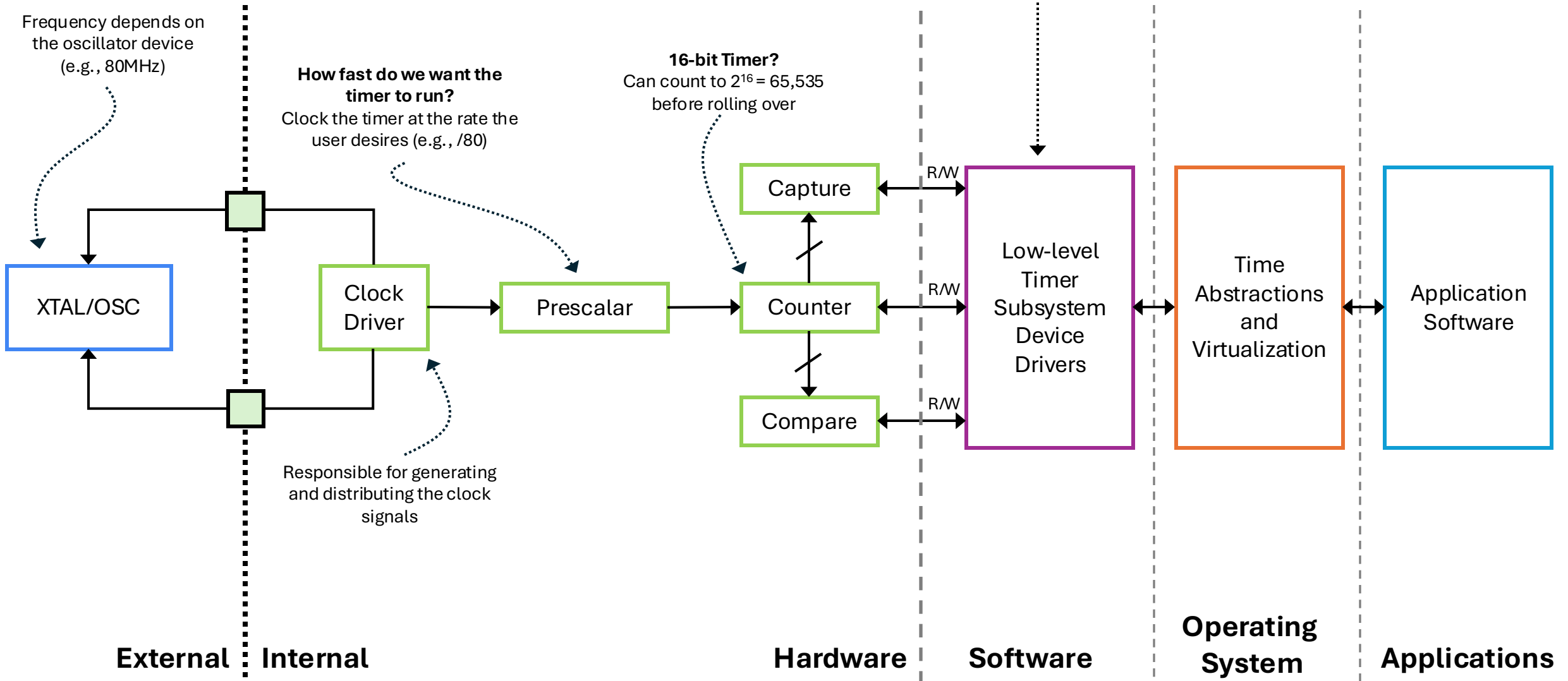
Event Synchronization – synchronize events internally or externally

Generate Timeouts – maximum wait time



How does a timer work?

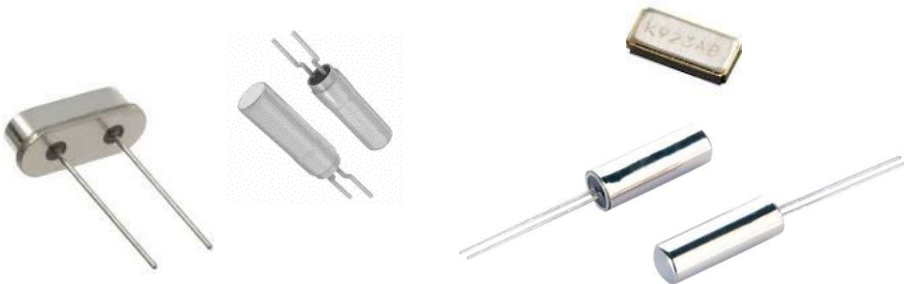
```
timer_tick:
    ldr r0, count;
    add r0, r0, #1;
    ...
```



Types of Clock Sources

Quartz Crystal

- Typically 32.768 KHz for Real Time Clocks
 - Used to track time
 - Years, months, days, hours, mins, seconds
- Can have much higher frequencies (MHz range)

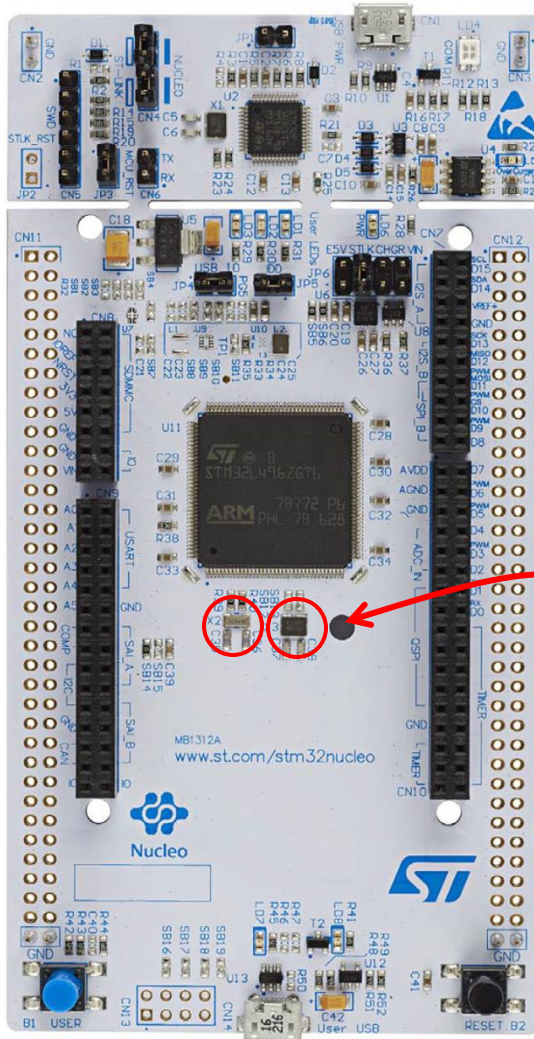


SAW Resonator

- Surface Acoustic Wave Resonator
- Typically 10– 100 MHz
- Limited limited number of frequencies



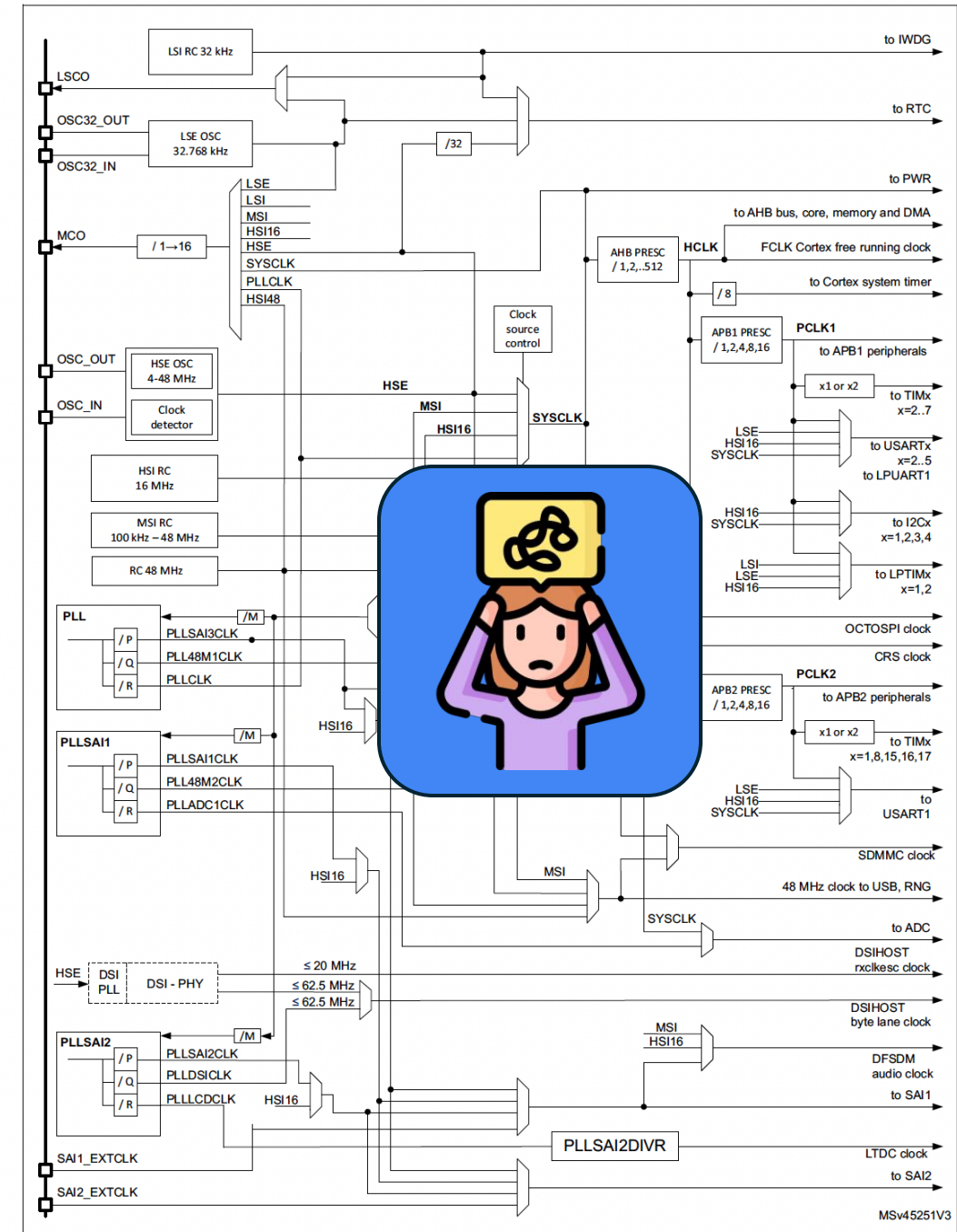
STM32 Clock Tree



External Crystals

From the
STM32 Manual

Figure 16. Clock tree for STM32L4Rxxx and STM32L4Sxxx devices



STM32 Clock Tree

Some Terminology

Low Speed External (LSE)

Low Speed Internal (LSI)

High Speed External (HSE)

High Speed Internal (HSI)

Multi-Speed Internal (MSI)

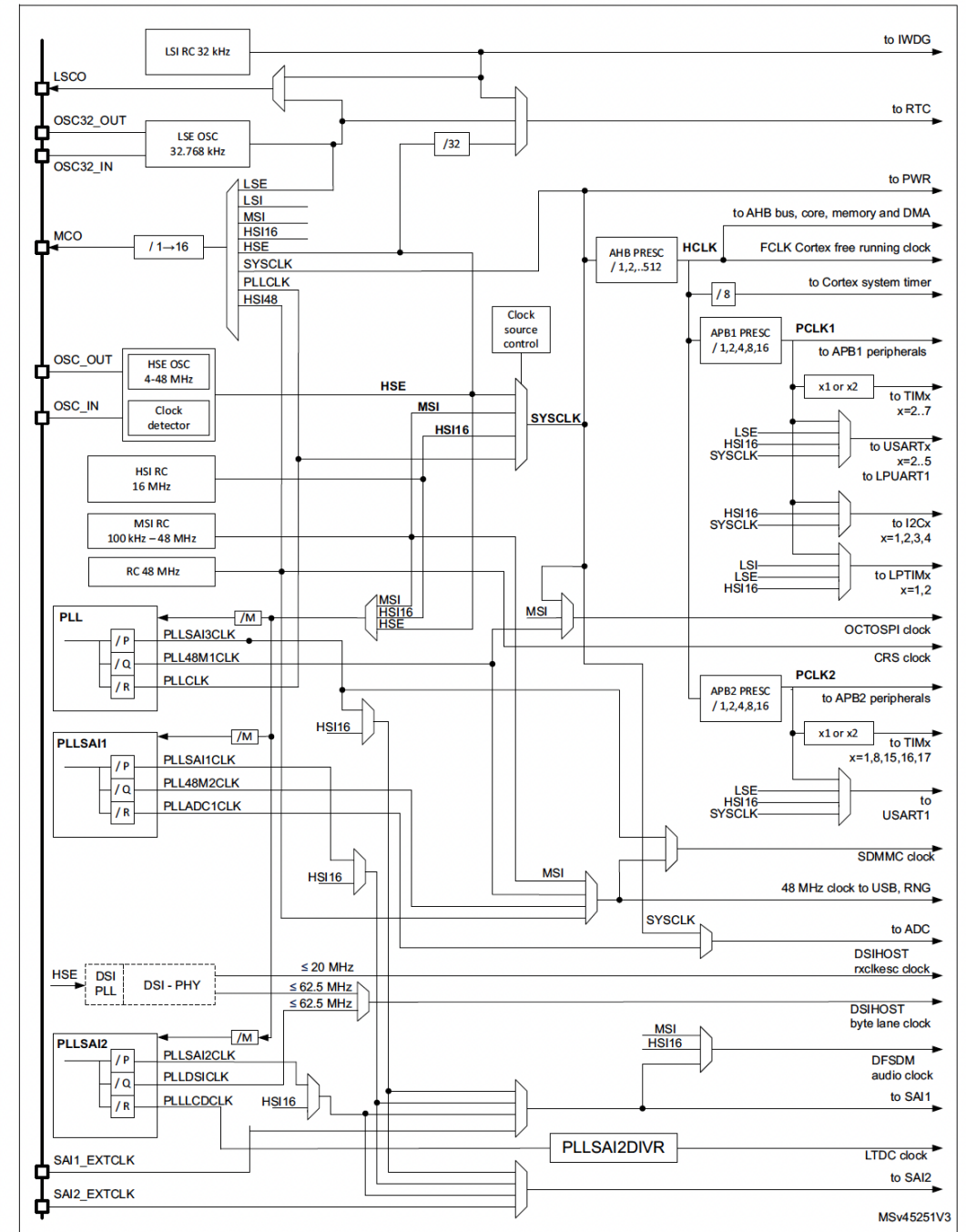
Output Clock (MCO)

Real Time Clock (RTC)

System Clock (SYSCLK)

Phase Locked Loop (PLL)

Figure 16. Clock tree for STM32L4Rxxx and STM32L4Sxxx devices



STM32 Clock Tree

Select what clock source should be used for SYSCLK

Prescalars

Low Speed Internal Resistor-Capacitor (RC) 32kHz Oscillator

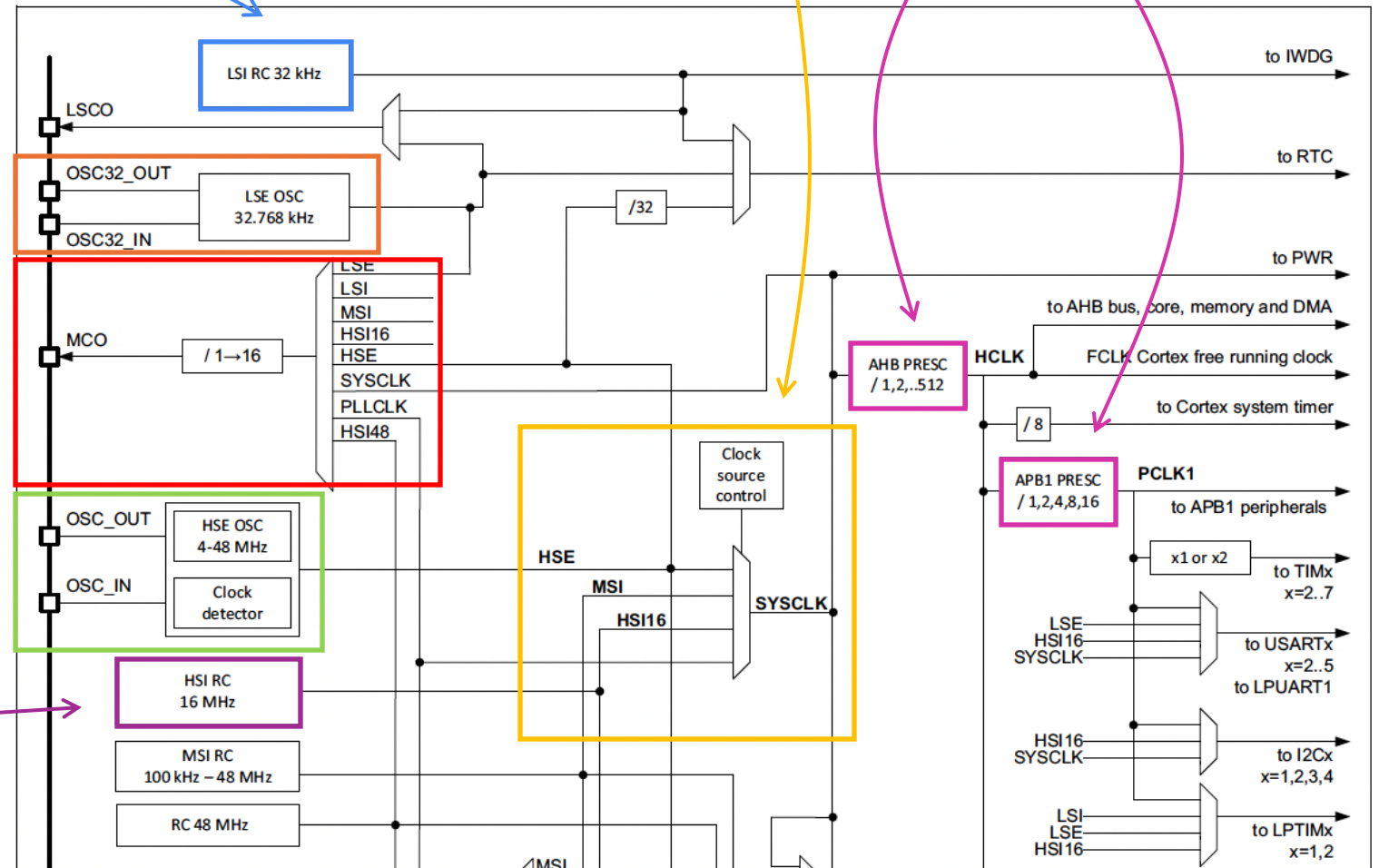
Low Speed External 32.768 kHz Crystal Oscillator (X2 = 32.768kHz)

Microcontroller Clock Output

High Speed External Oscillator (X3 = 8MHz)

High Speed Internal RC 16MHz Oscillator

Figure 16. Clock tree for STM32L4Rxxx and STM32L4Sxxx devices



STM32 Clock Tree

Phase Locked Loop

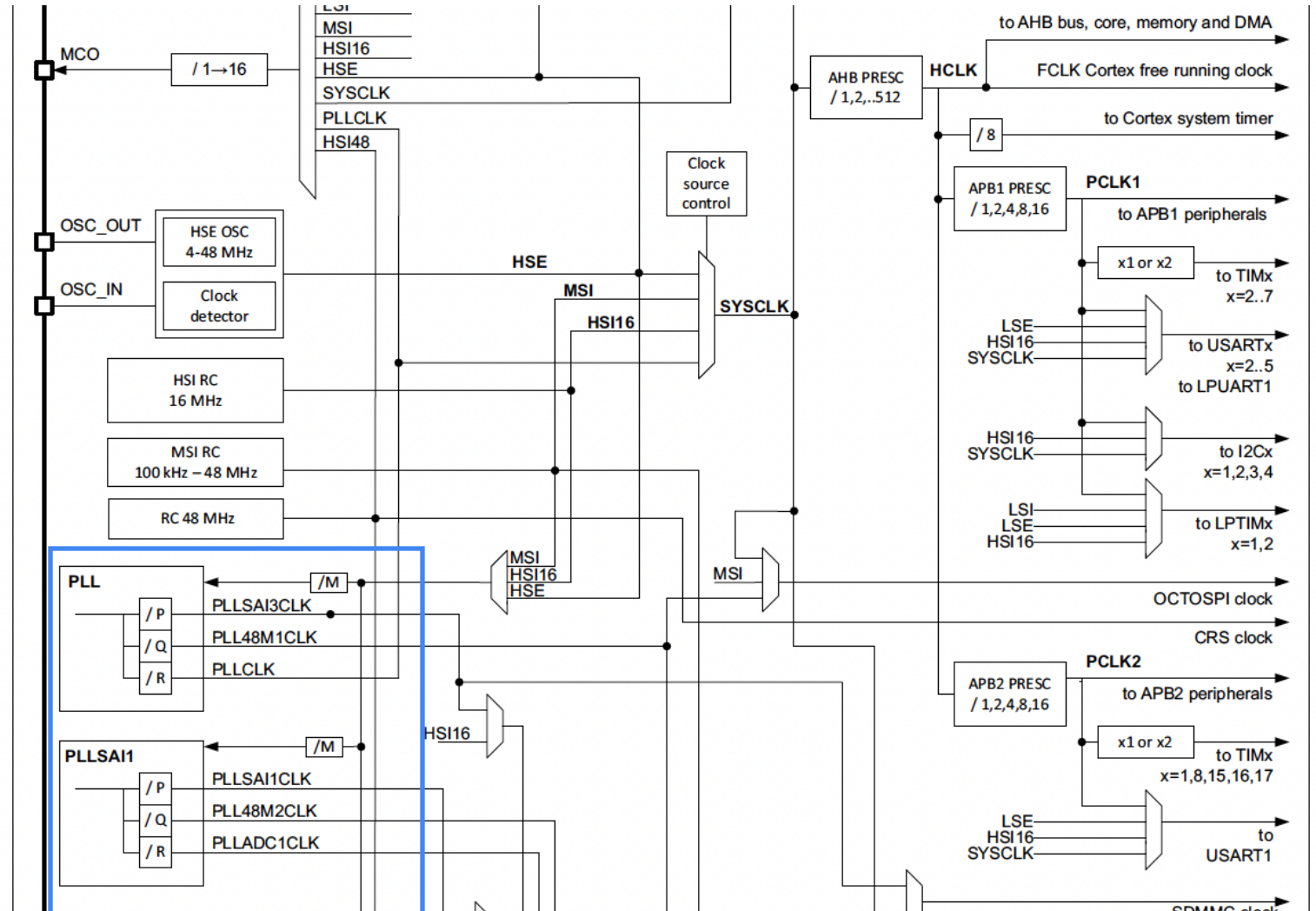
Generate a stable frequency that is a *multiple* of the input frequency

Higher clock noise

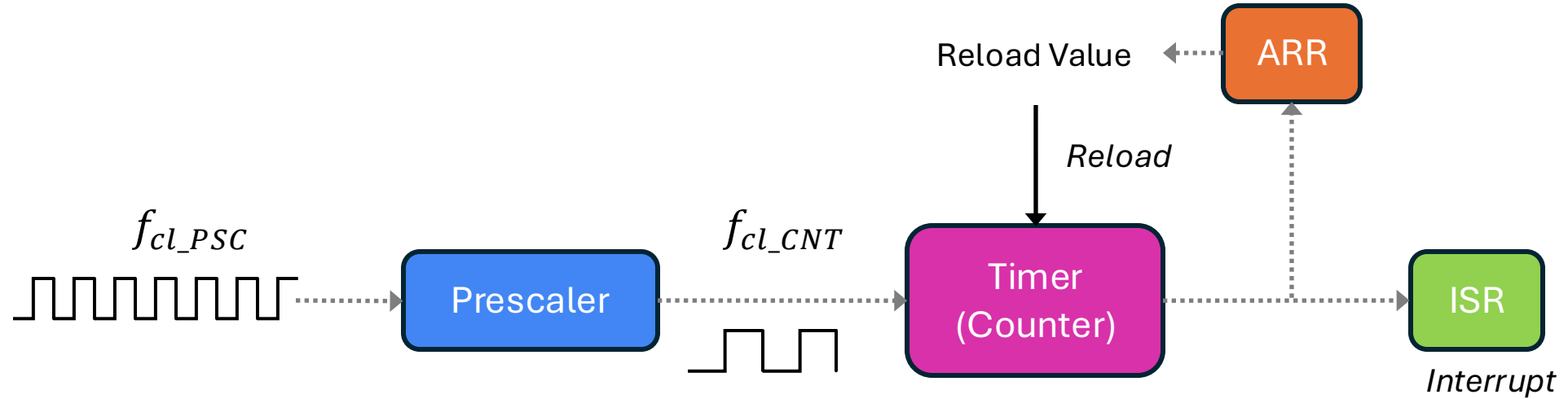
Increased power consumption

$$f = \frac{f_{PLLSource}}{M} \times N$$

(divide by /R for PLLCLK, etc.)



Counter Basics



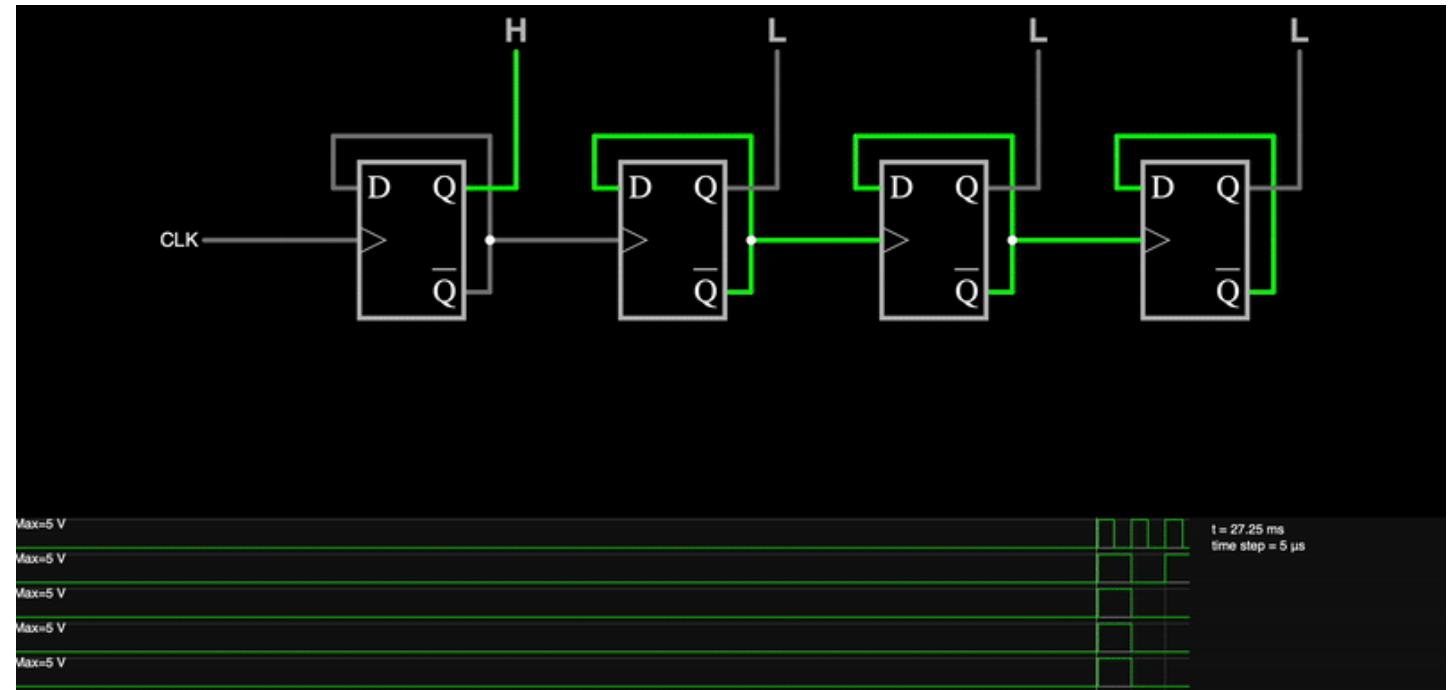
$$f_{cl_CNT} = \frac{f_{cl_PSC}}{PSC + 1}$$

Clock Divider

How would you implement a clock divider?

Use DFFs

Falstad Clock Divider
Example (LINK)



Clock Dividers vs. Prescaler on the STM32

Clock Dividers

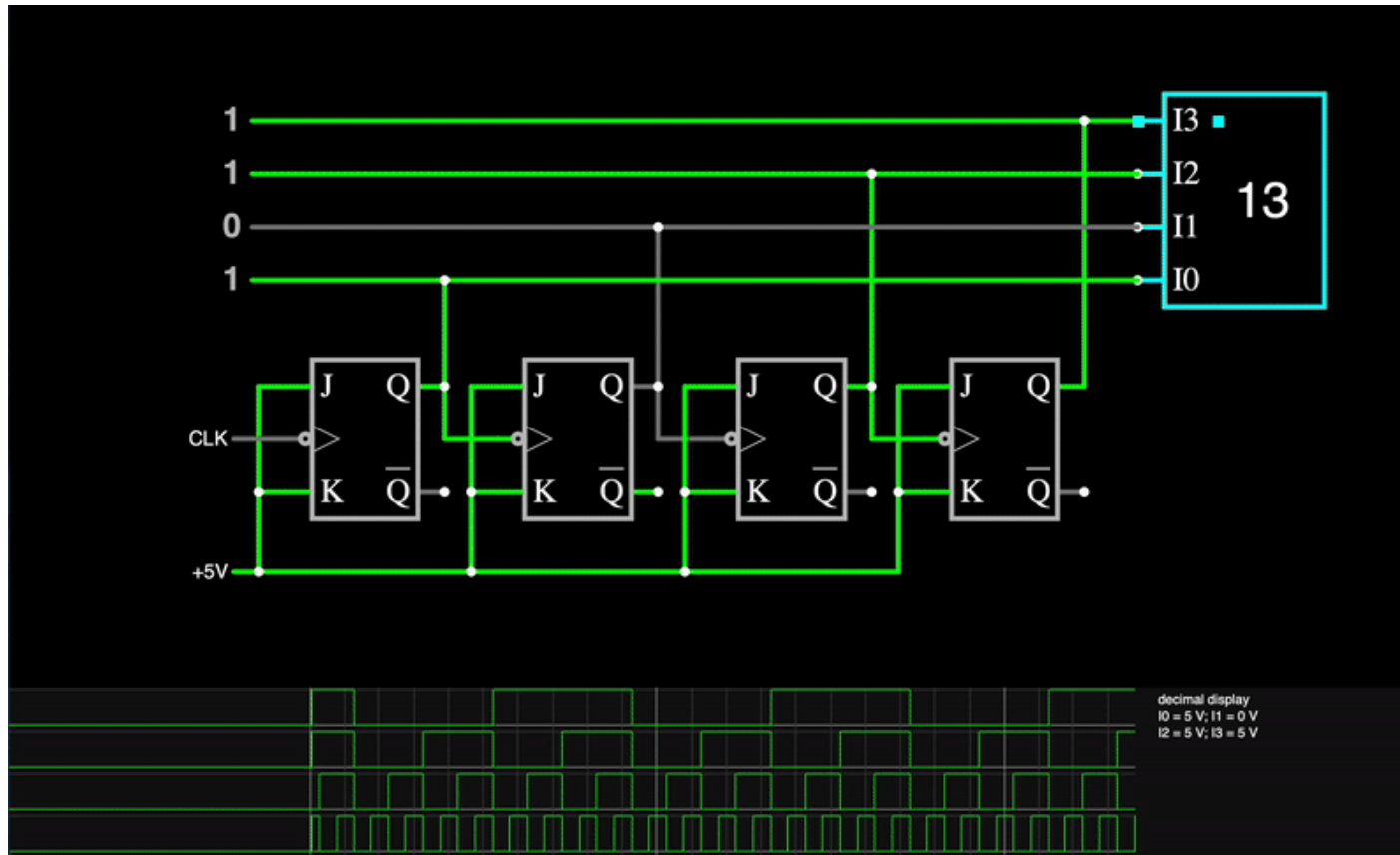
- Divide the clock frequency by powers of 2
- Use few DFFs
- Fixed values: 2/ 4/ /8

Prescalers

- Divide the counter clock frequency by any factor between 1 and 65536
- Based on 16-bit counter
- Controlled through a 16-bit register

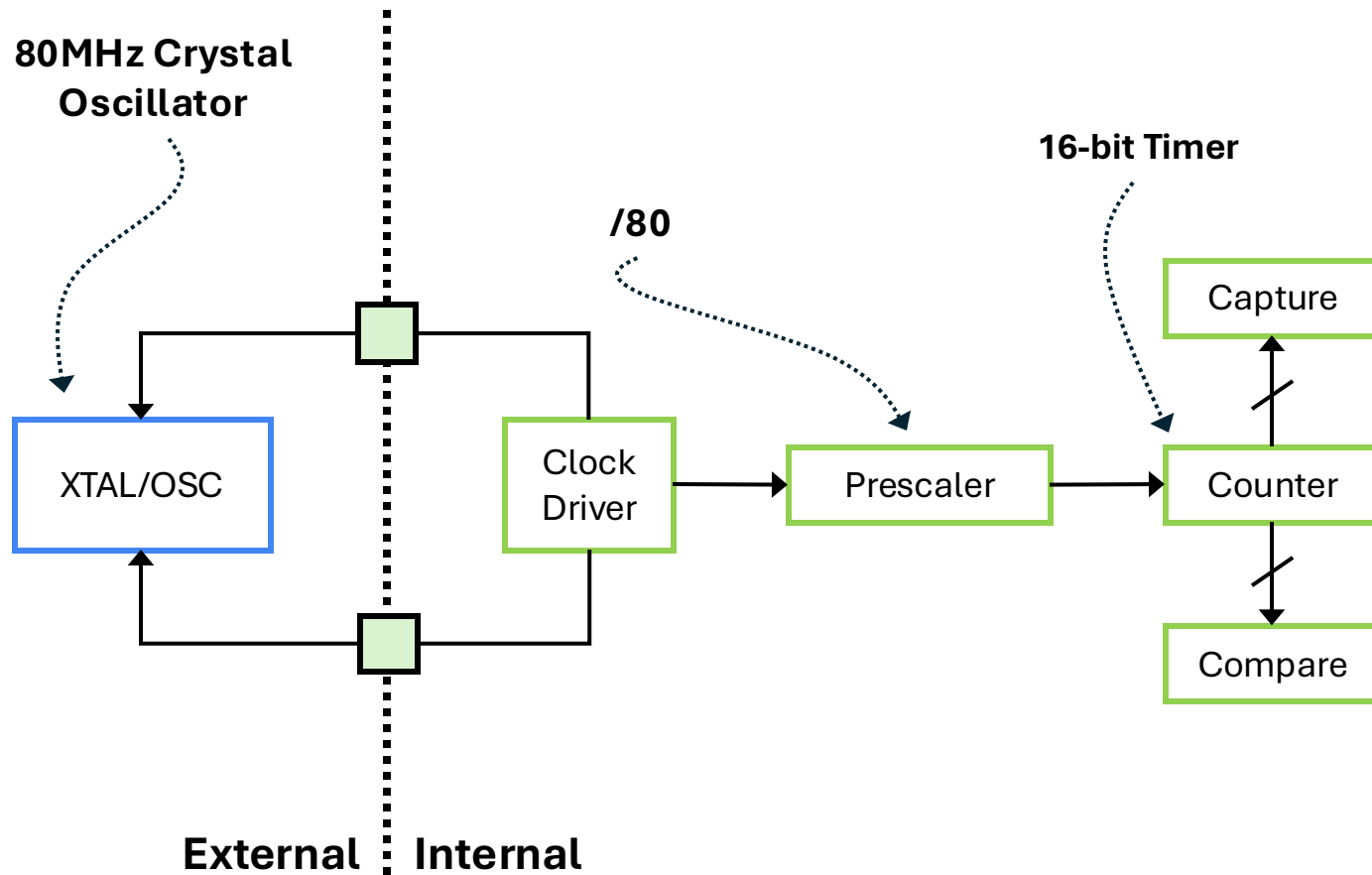
Counter Example

[Falstad Counter Example \(LINK\)](#)



- Count up
- Count down
- Hold

Simple Example



What is the output clock frequency after the prescaler?

1MHz

What is the longest duration of event that could be measured?

16-bit Timer?

Can count to $2^{16} = 65,535$ before rolling over

$$\frac{1}{f} = \frac{1}{1\text{MHz}} = 0.001\text{ ms} \times 65535 = 65.535\text{ ms}$$

Or measure events no longer than 65.5ms!

Oscillating Frequency Tradeoffs

There is a tradeoff between **resolution** and **range**

80MHz OSC Resolution:

$$\frac{1}{80e6} = 12.5 \text{ ns}$$

1MHz OSC Resolution:

$$\frac{1}{1e6} = 1000 \text{ ns}$$

16-bit Timer

80MHz OSC Range:

$$\frac{1}{80e6} \times 2^{16} = 0.8192 \text{ ms}$$

1MHz OSC Range:

$$\frac{1}{1e6} \times 2^{16} = 65.536 \text{ ms}$$

Key Takeaways

Higher oscillating frequency = higher resolution

Higher oscillating frequency = Timer overflows quicker

Capture and Compare

Capture and Compare

Two primary timer modes:

1

Measure how long something takes

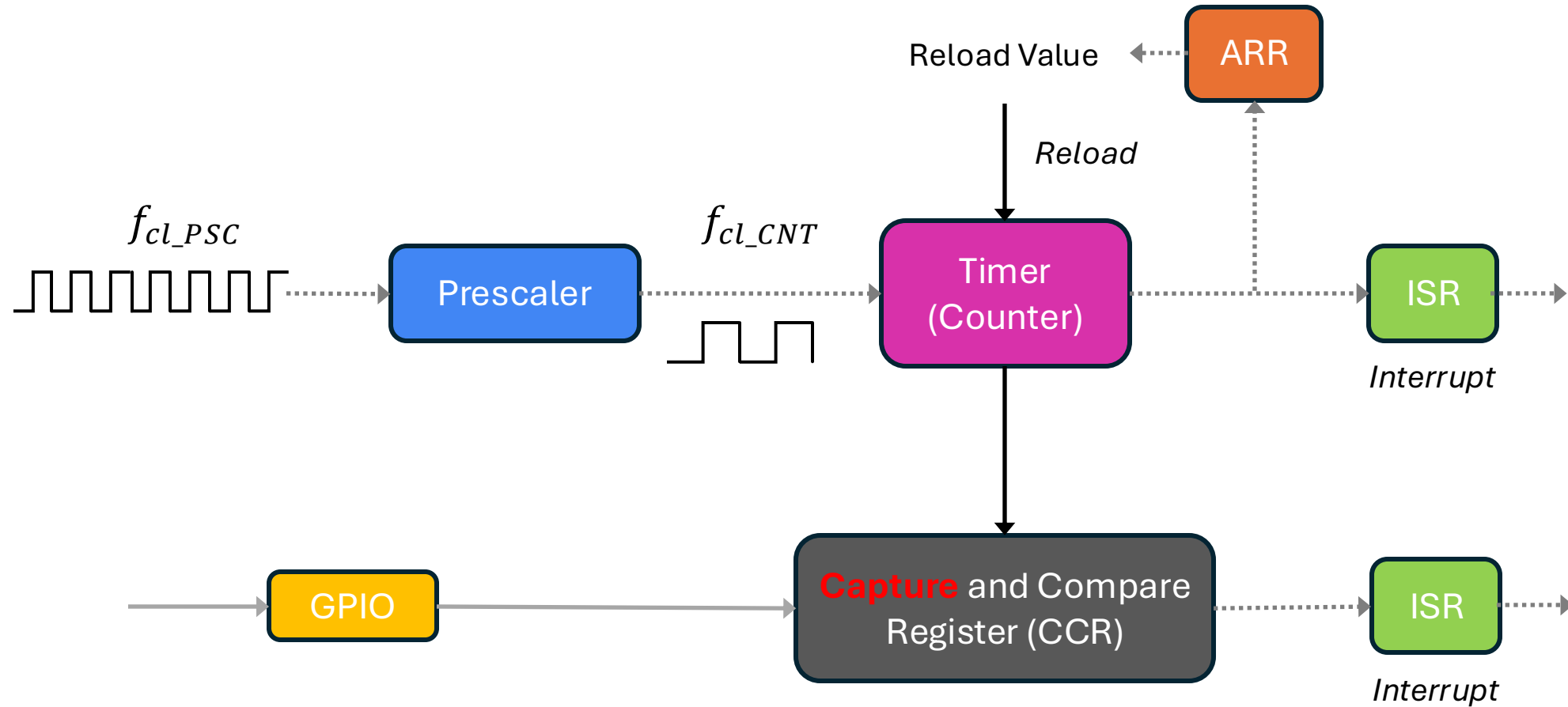
capture how long something takes to happen

2

Have something happen once or every N time periods

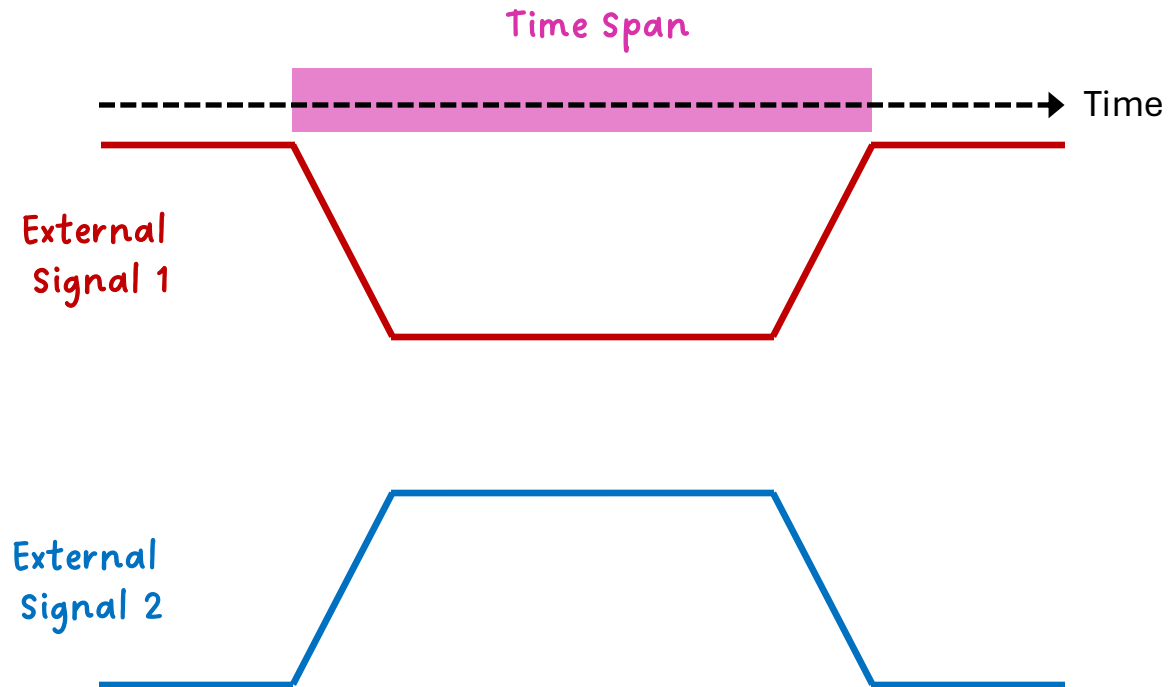
compare a clock source to desired time and act

Capture

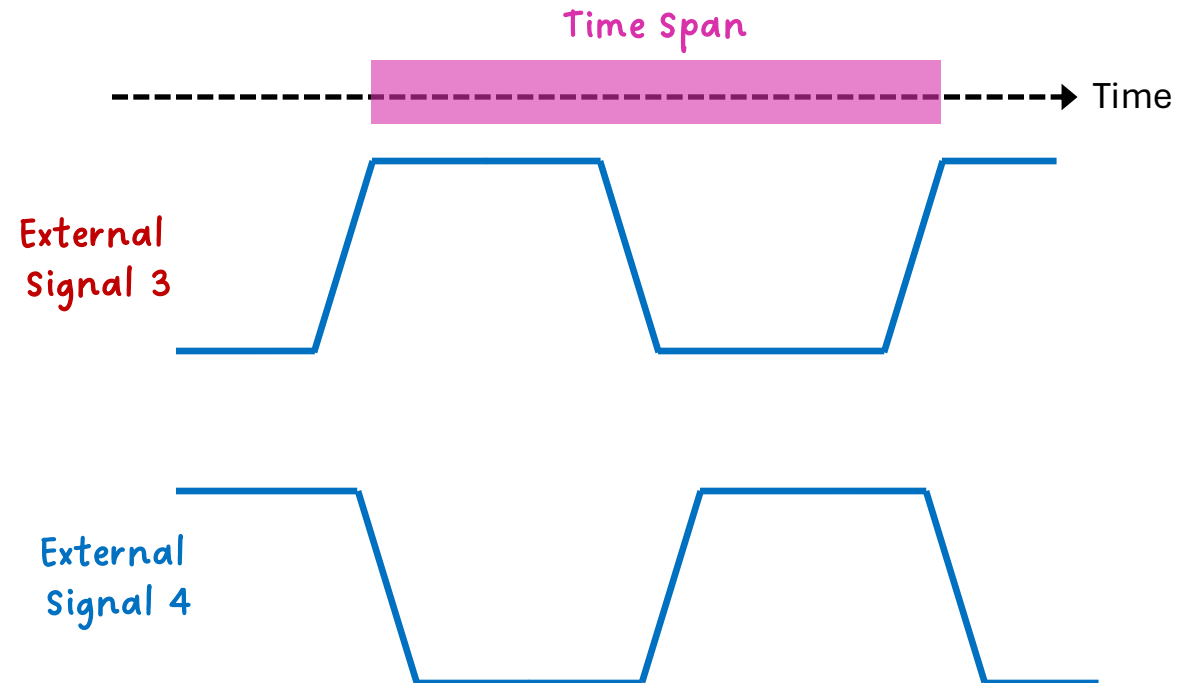


Input Capture

Monitor **Both** Rising & Falling Edge



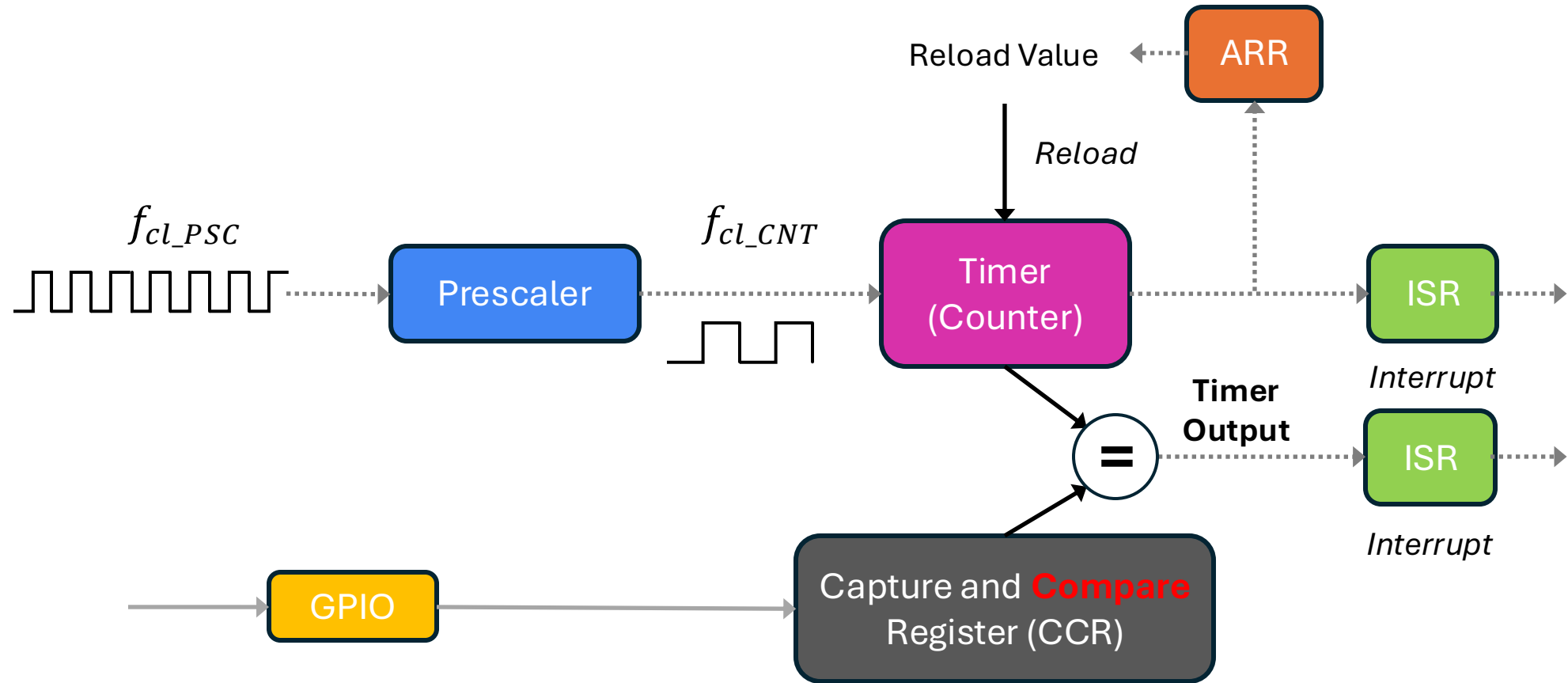
Monitor **Only** Rising or Falling Edge



Input Filtering



Compare



Output Compare Modes

Output Compare Timing

Comparison between the output compare register and the counter does not trigger a change in the output.

Output Compare Active

Set the channel output to active level on match

Output Compare Inactive

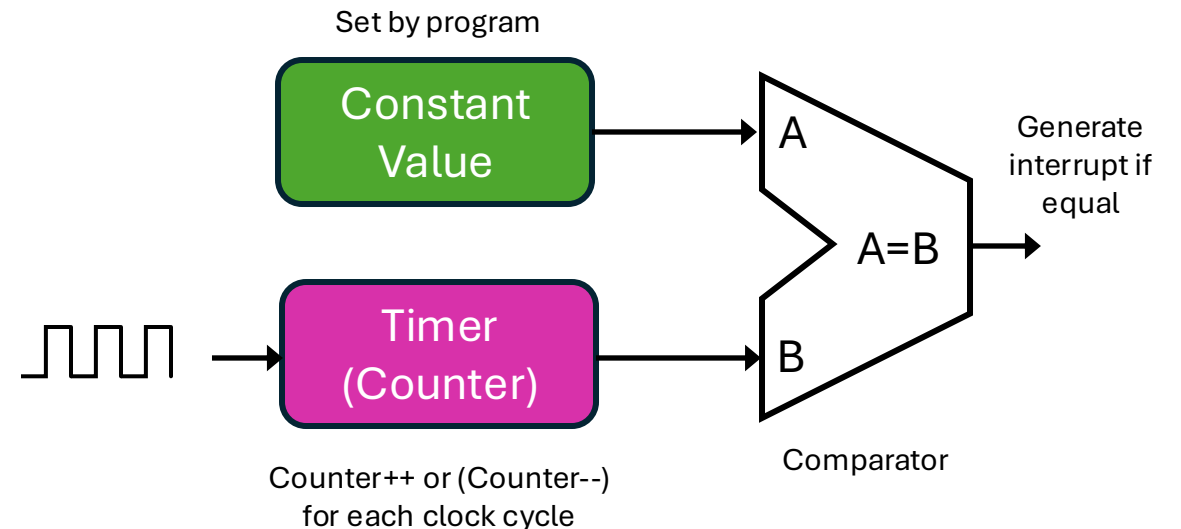
Set the channel output to inactive level on match

Output Compare Toggle

Toggle the output channel when the capture/compare registers match

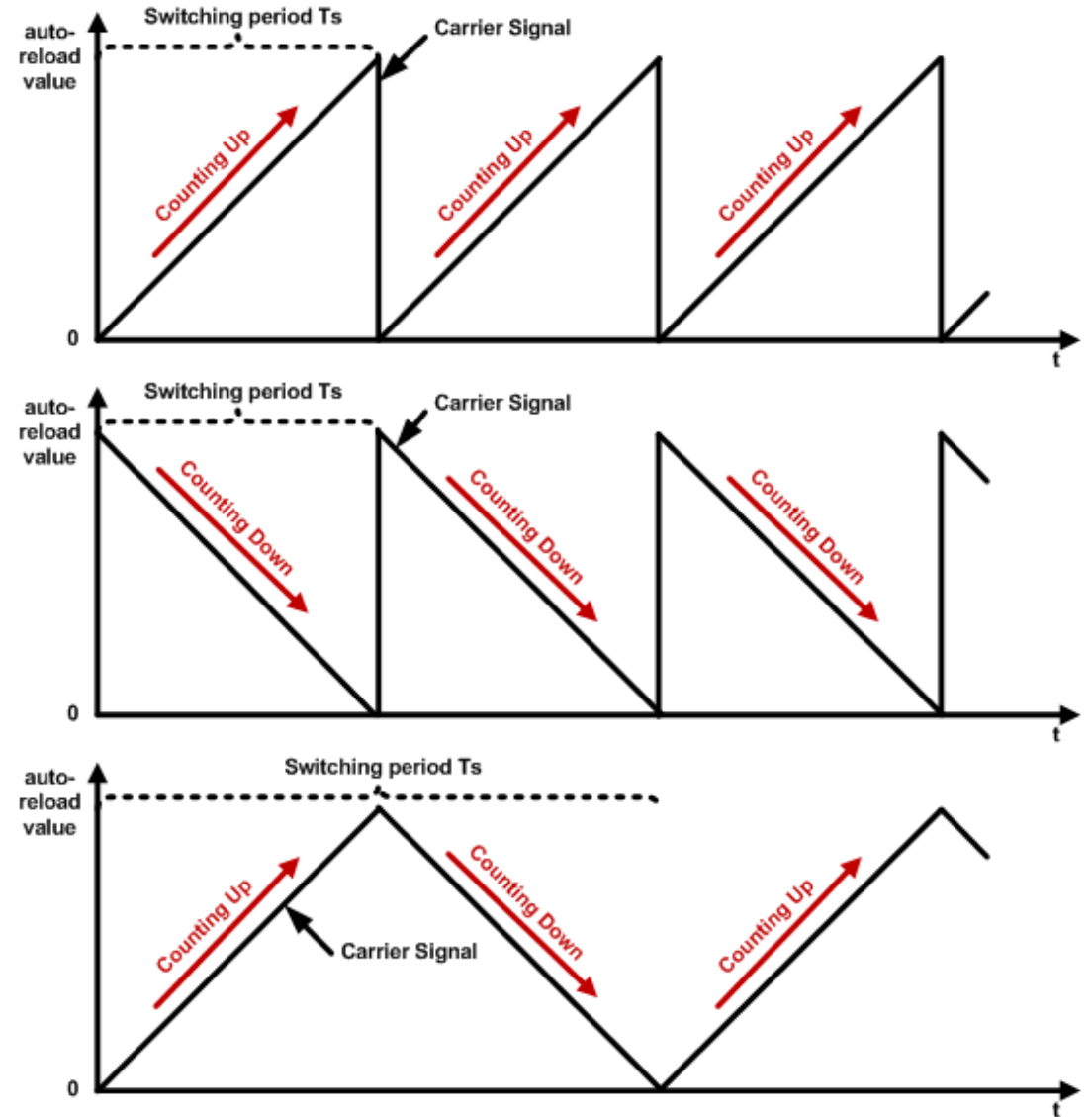
Output Compare Forced Active/Inactive

OCREF forced high or low independently from the counter value

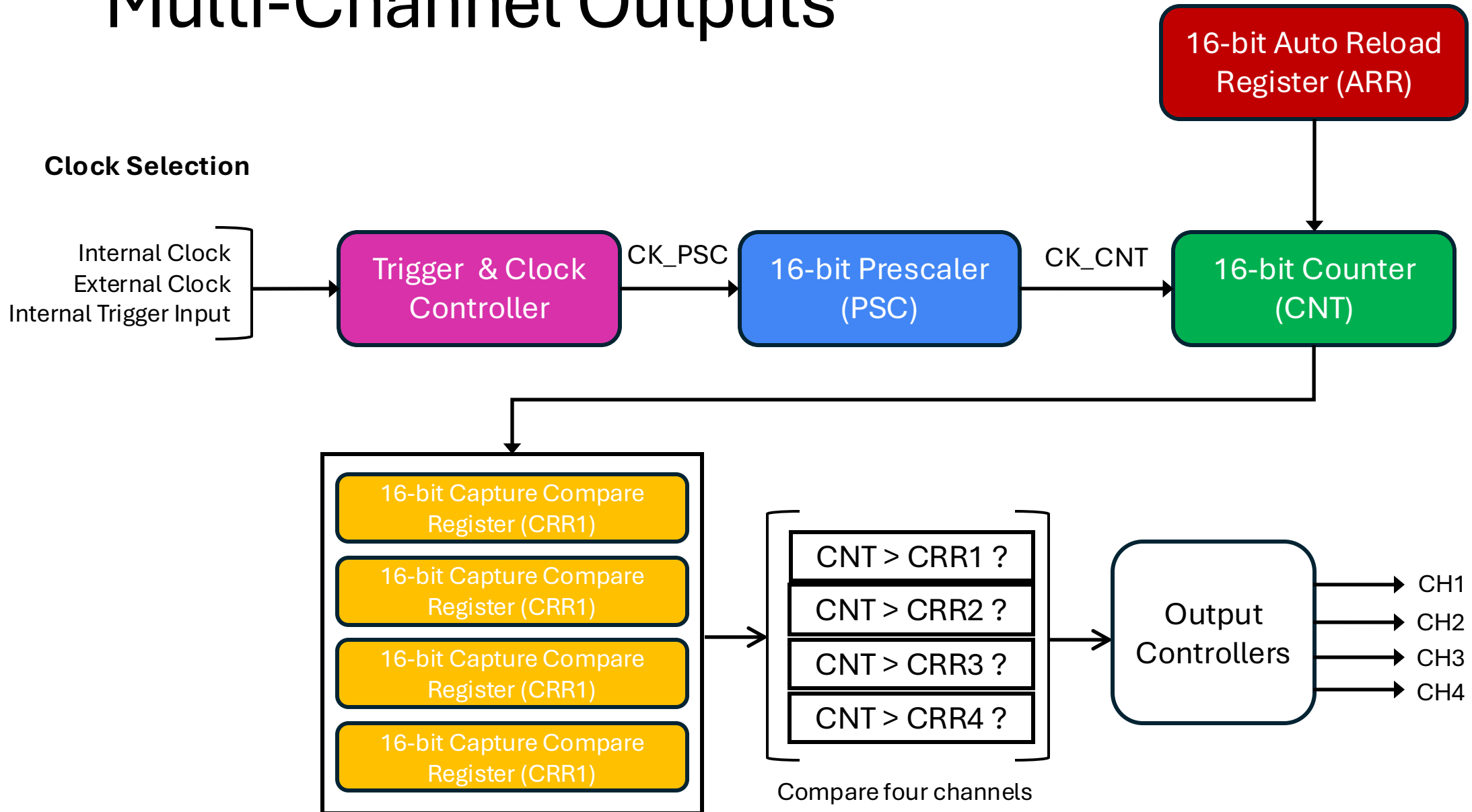


Counting Configurations

- Count up and reload
- Count down and reload
- Count up and then count down



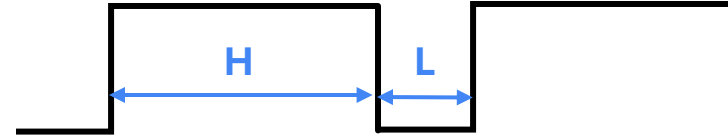
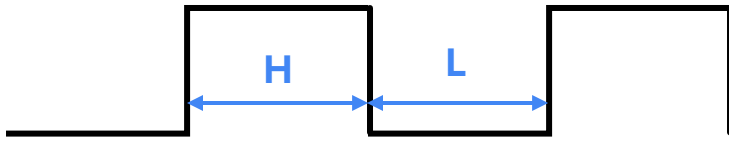
Multi-Channel Outputs



Pulse Width Modulation

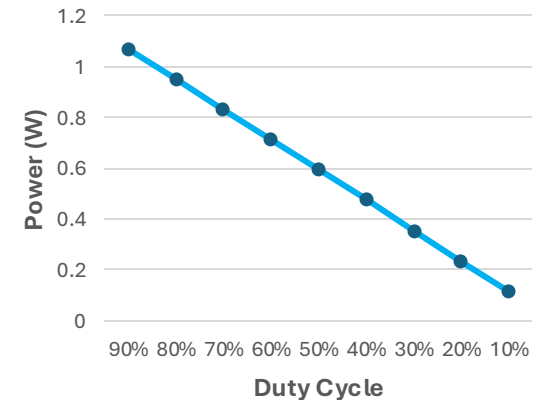
Pulse Width Modulation (PWM)

Basic Idea: Create a digital output wave of fixed frequency, but allow the microcontroller to vary its duty cycle



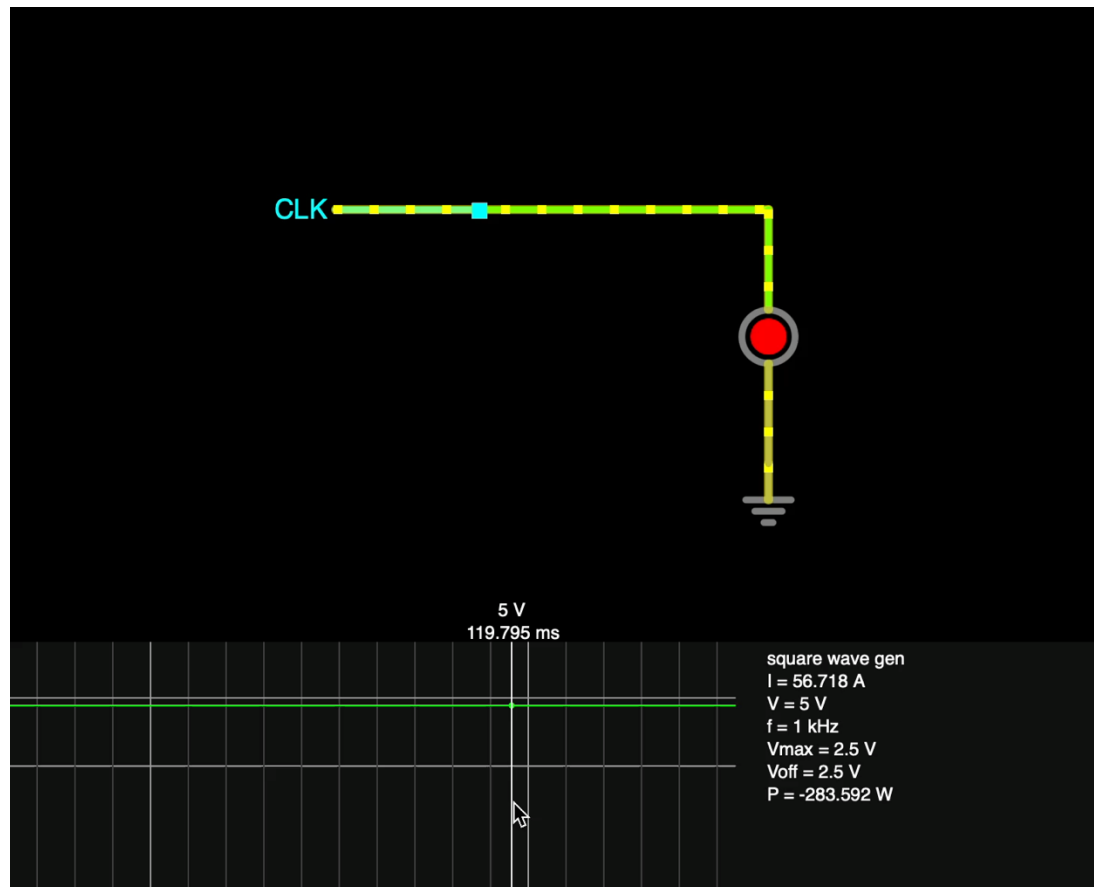
$$\text{duty cycle} = \frac{H}{H + L} \quad P = V \times I$$

$$\text{Delivered Power} = P * (\text{duty cycle}) = \frac{H}{H + L} \times P$$

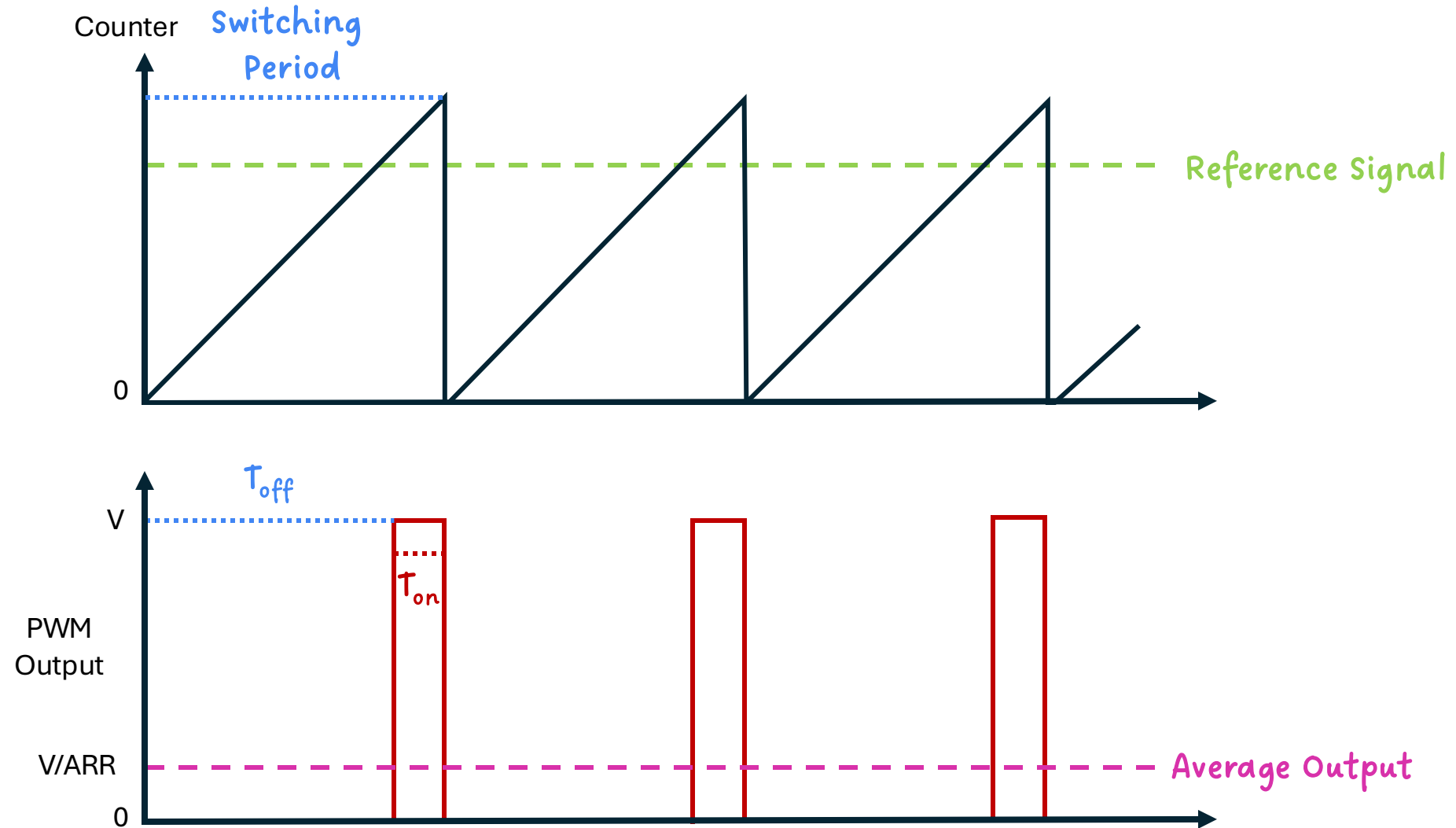


Pulse Width Modulation

High level example

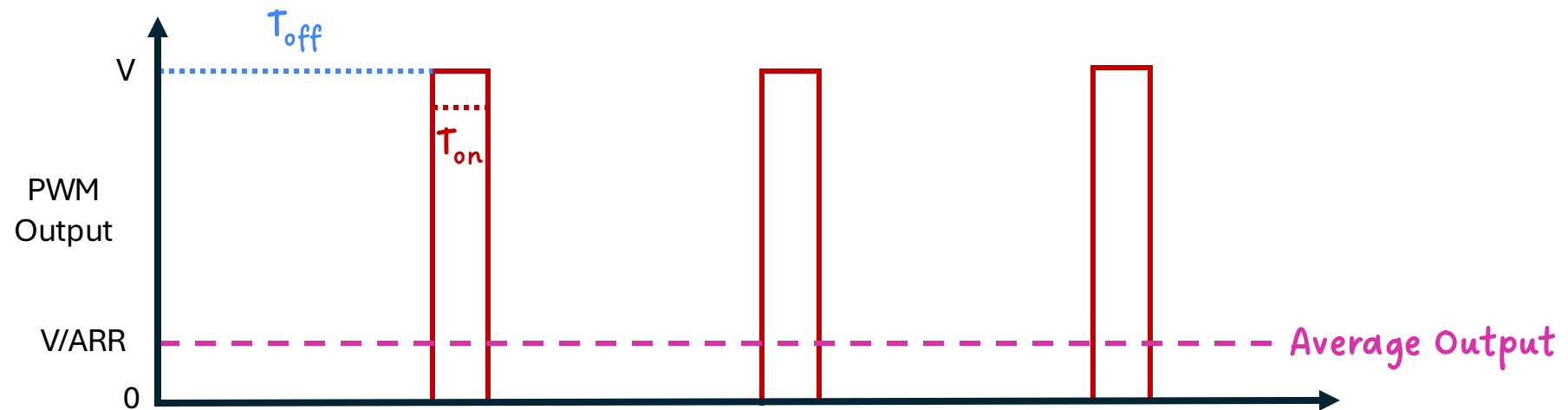
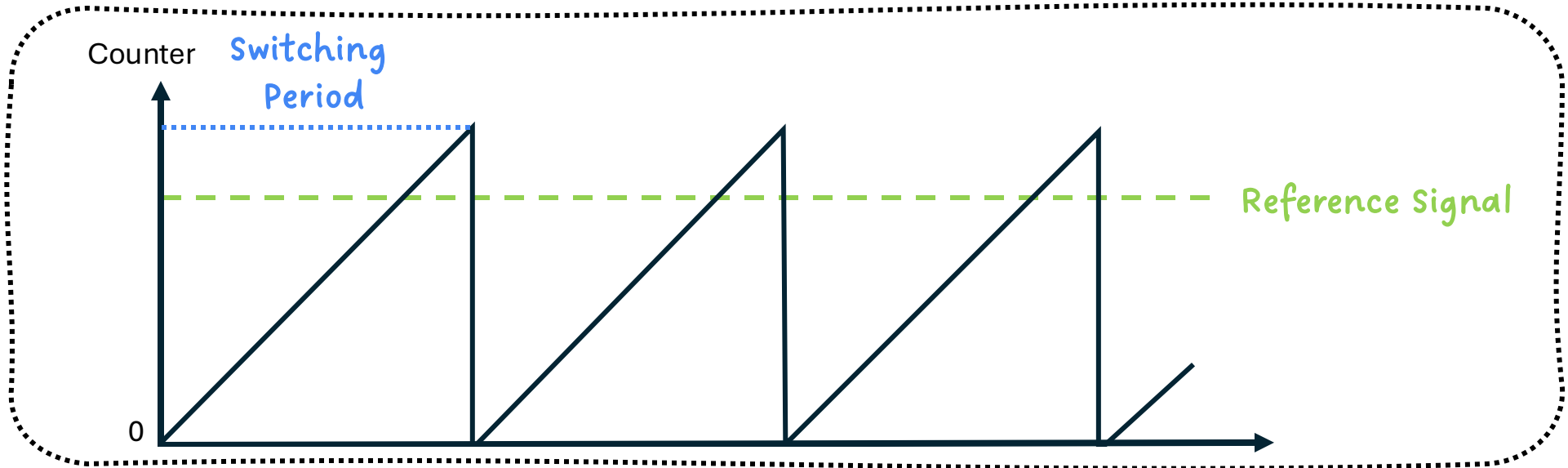


PWM Mode

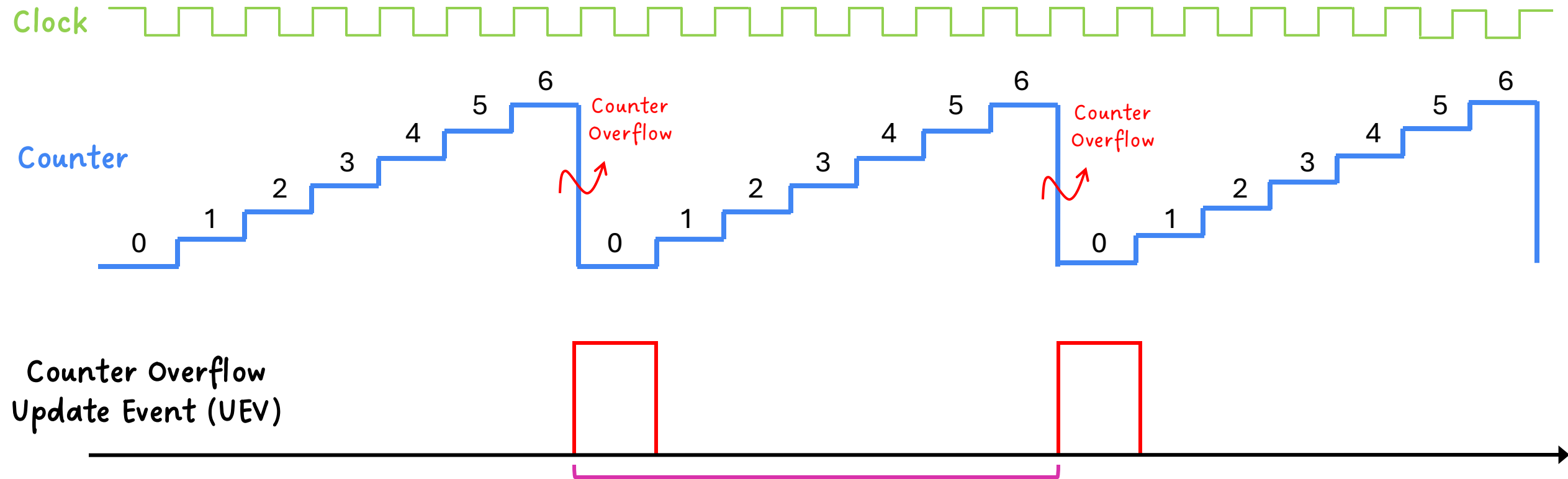


PWM Mode

How do we
setup the
counter?



Edge-aligned Mode (Up-counting)



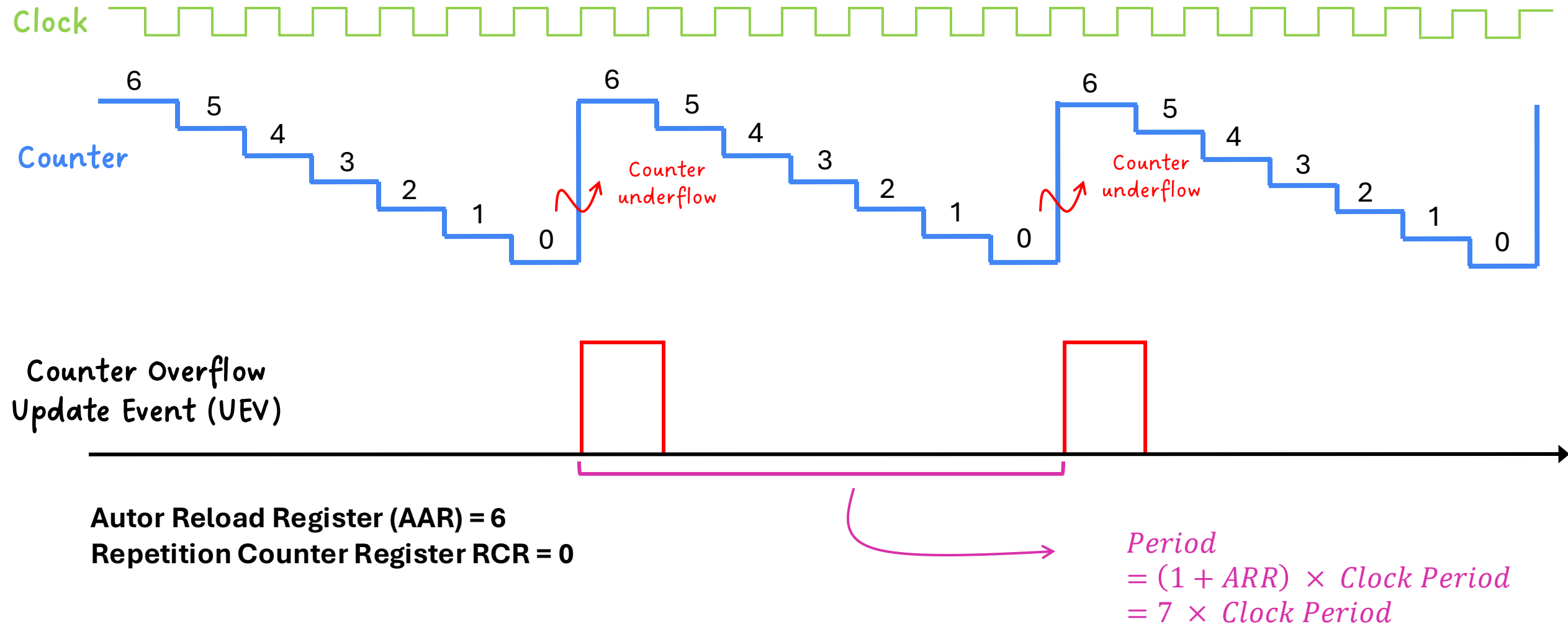
Autoreload Register (ARR) = 6

Repetition Counter Register RCR = 0

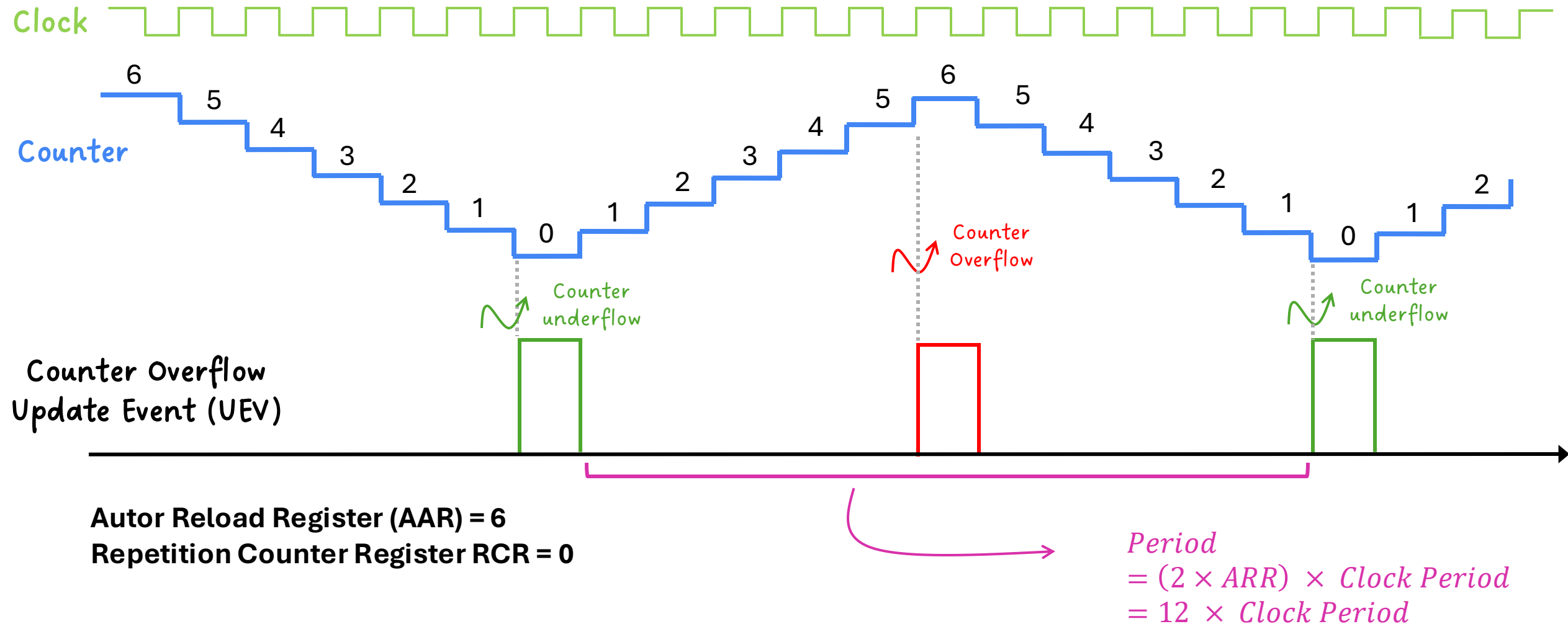
0 = Repeat forever. Set to a non-zero value, the timer counts the number of occurrences of an update event before generating an interrupt

$$\begin{aligned} \text{Period} &= (1 + \text{ARR}) \times \text{Clock Period} \\ &= 7 \times \text{Clock Period} \end{aligned}$$

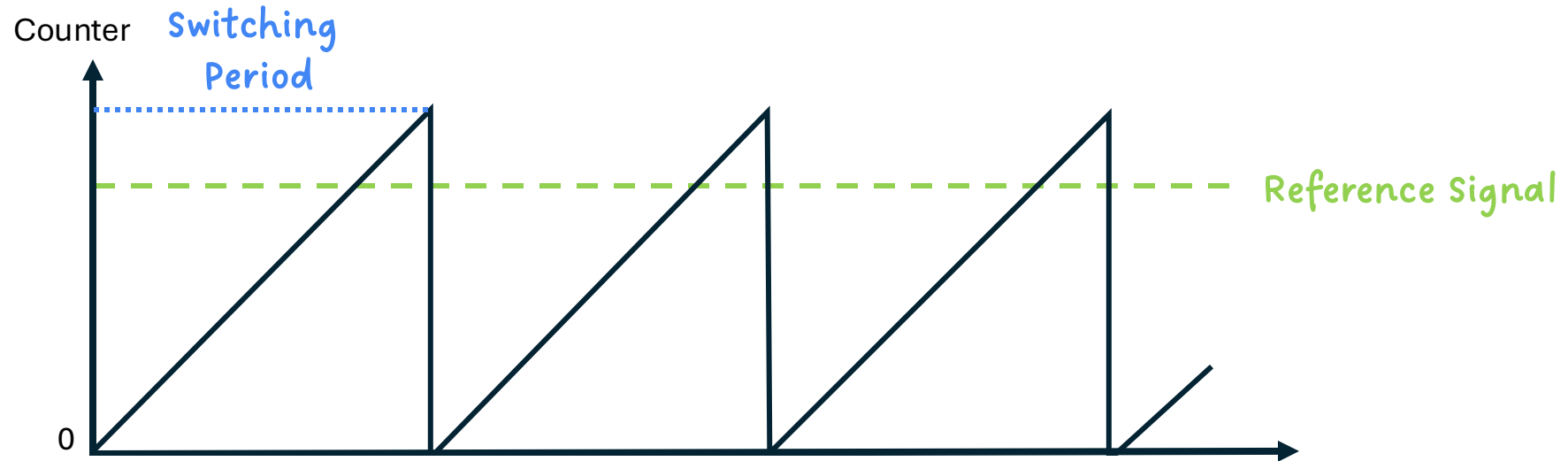
Edge-aligned Mode (Down-counting)



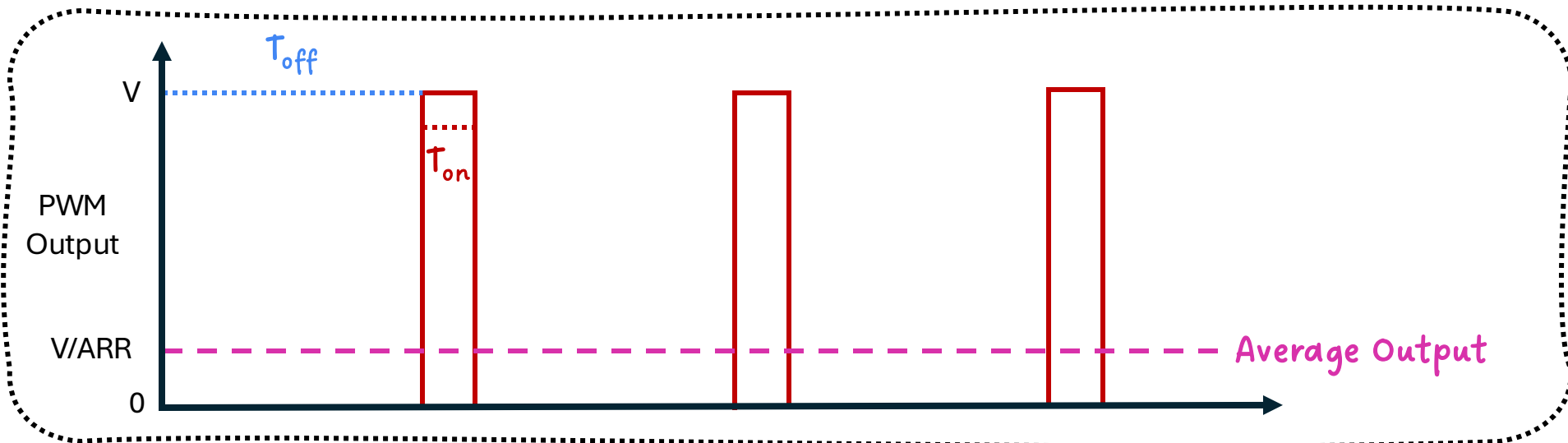
Center-aligned Mode



PWM Mode

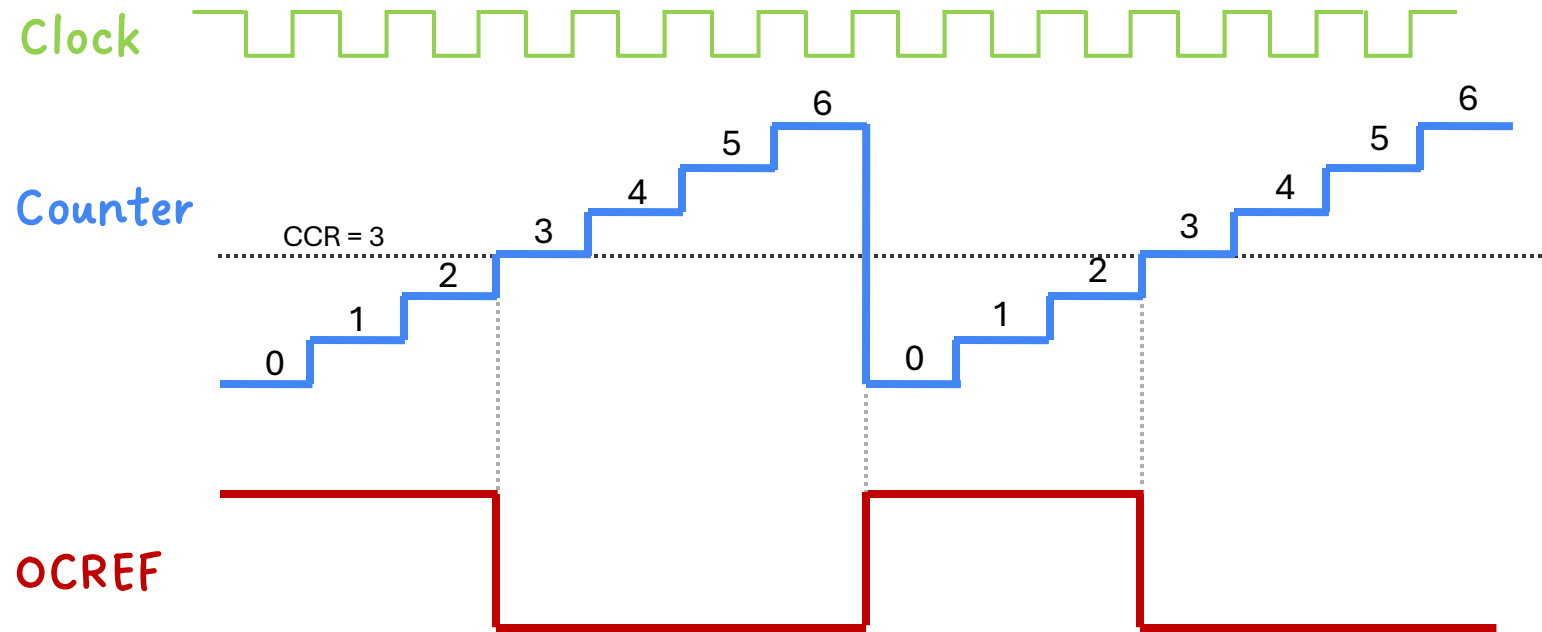


How do we
generate
the PWM
output
signal?



PWM Mode 1 (Low True)

Up-counting Mode, ARR = 6, CCR = 3, RCR = 0



Timer Output

High if counter < CCR

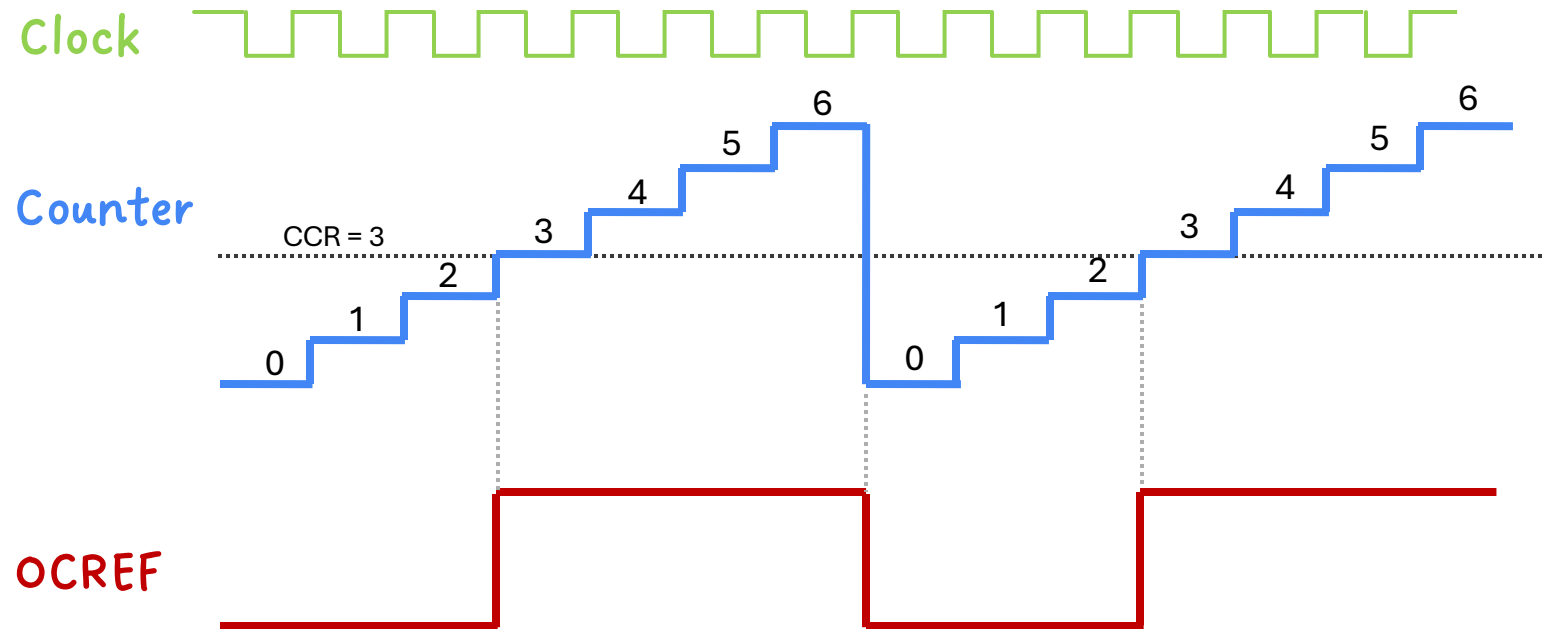
Low if counter $\geq$ CCR

$$\text{Duty Cycle} = \frac{CCR}{ARR + 1} = \frac{3}{7}$$

$$\begin{aligned} \text{Period} &= (1 + ARR) \times \text{Clock Period} \\ &= 7 \times \text{Clock Period} \end{aligned}$$

PWM Mode 2 (High True)

Up-counting Mode, ARR = 6, CCR = 3, RCR = 0



Timer Output

Low if counter < CCR

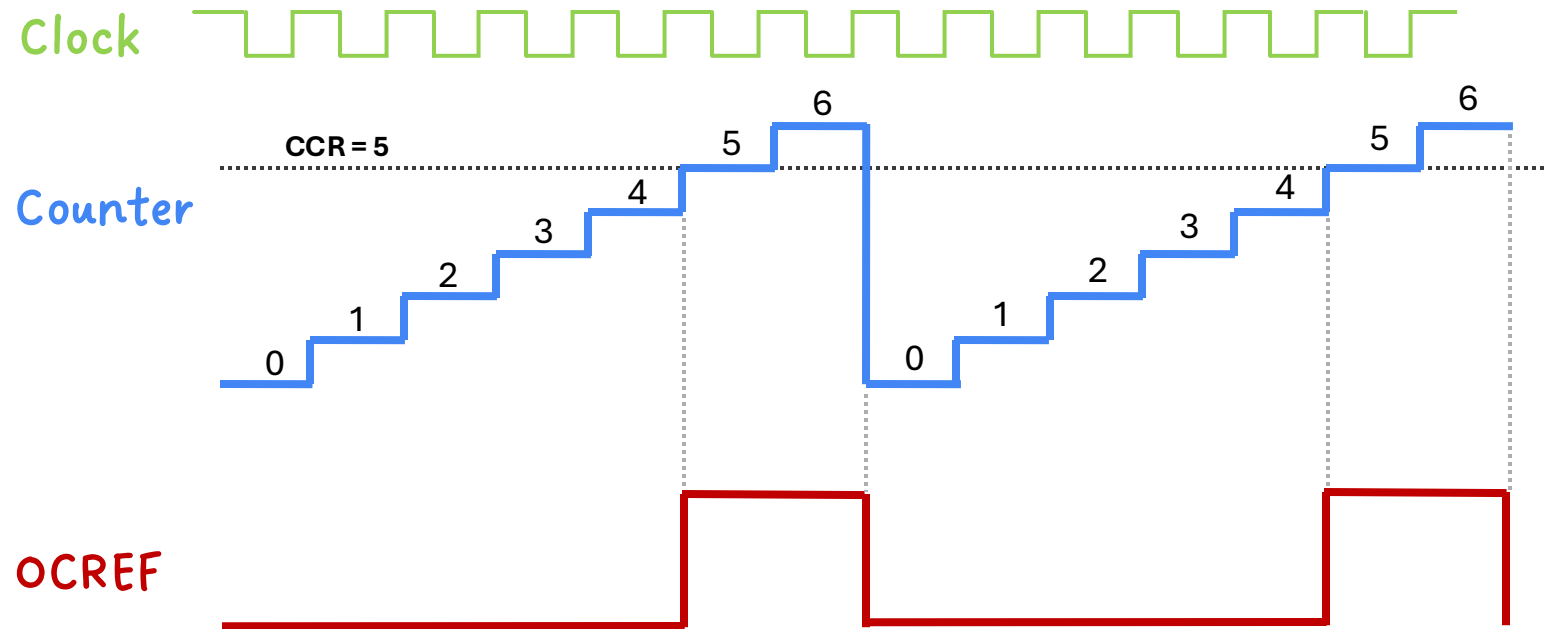
High if counter $\geq$ CCR

$$\text{Duty Cycle} = 1 - \frac{CCR}{ARR + 1} = \frac{4}{7}$$

$$\begin{aligned} \text{Period} &= (1 + ARR) \times \text{Clock Period} \\ &= 7 \times \text{Clock Period} \end{aligned}$$

PWM Mode 2 (High True)

Up-counting Mode, ARR = 6, **CCR = 5**, RCR = 0



Timer Output

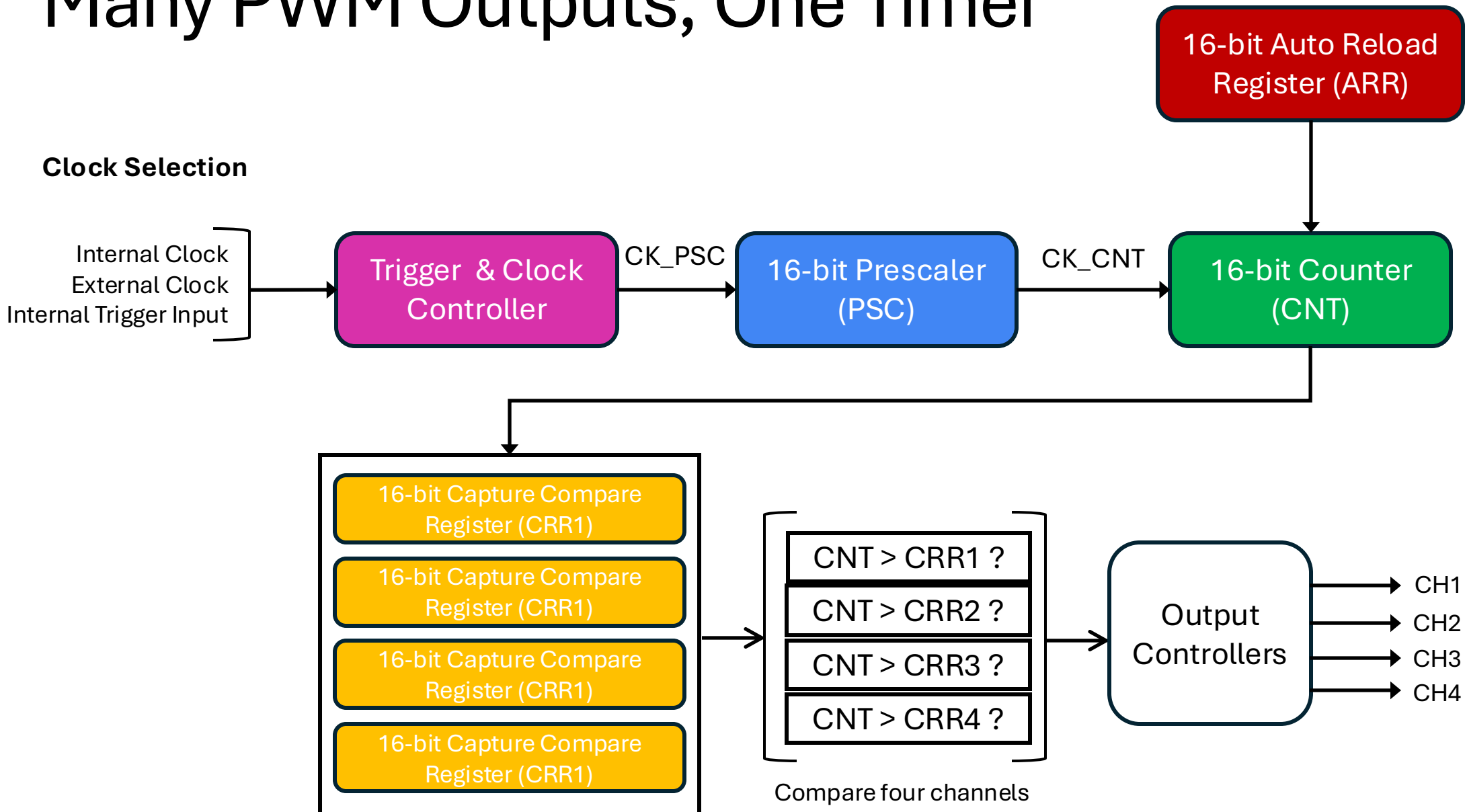
Low if counter < CCR

High if counter $\geq$ CCR

$$\text{Duty Cycle} = 1 - \frac{CCR}{ARR + 1} = \frac{2}{7}$$

$$\begin{aligned} \text{Period} &= (1 + ARR) \times \text{Clock Period} \\ &= 7 \times \text{Clock Period} \end{aligned}$$

Many PWM Outputs, One Timer



Read the Datasheet

We covered many things, but not everything!

Table 278. Output control bits for complementary OCx and OCxN channels with break feature

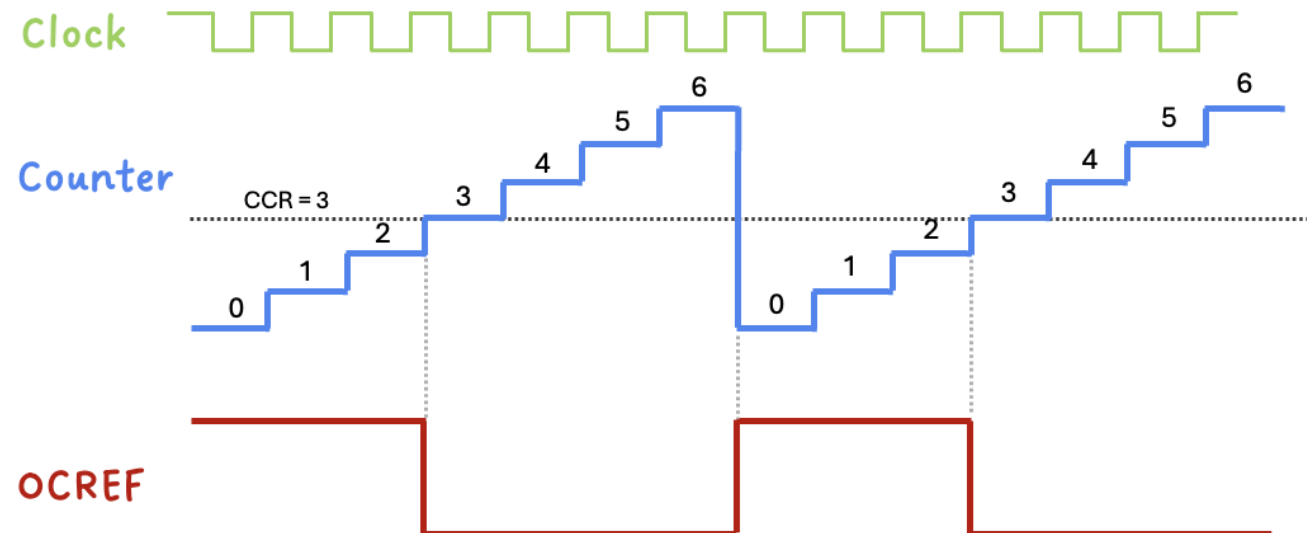
Control bits					Output states ⁽¹⁾	
MOE bit	OSSI bit	OSSR bit	CCxE bit	CCxNE bit	OCx output state	OCxN output state
1	X	X	0	0	Output disabled (not driven by the timer: Hi-Z) OCx=0, OCxN=0	
		0	0	1	Output disabled (not driven by the timer: Hi-Z) OCx=0	OCxREF + Polarity OCxN = OCxREF xor CCxNP
		0	1	0	OCxREF + Polarity OCx=OCxREF xor CCxP	Output Disabled (not driven by the timer: Hi-Z) OCxN=0
		X	1	1	OCREF + Polarity + dead-time	Complementary to OCREF (not OCREF) + Polarity + dead-time
		1	0	1	Off-State (output enabled with inactive state) OCx=CCxP	OCxREF + Polarity OCxN = OCxREF x or CCxNP
		1	1	0	OCxREF + Polarity OCx=OCxREF xor CCxP	Off-State (output enabled with inactive state) OCxN=CCxNP
0	0	X	X	X	Output disabled (not driven by the timer: Hi-Z).	
	0		0			
	0		1	Off-State (output enabled with inactive state) Asynchronously: OCx=CCxP, OCxN=CCxNP (if BRK or BRK2 is triggered).		
	1		0			
	1		1	Then (this is valid only if BRK is triggered), if the clock is present: OCx=OISx and OCxN=OISxN after a dead-time, assuming that OISx and OISxN do not correspond to OCx and OCxN both in active state (may cause a short circuit when driving switches in half-bridge configuration). Note: BRK2 can only be used if OSSI = OSSR = 1.		

Group Exercise

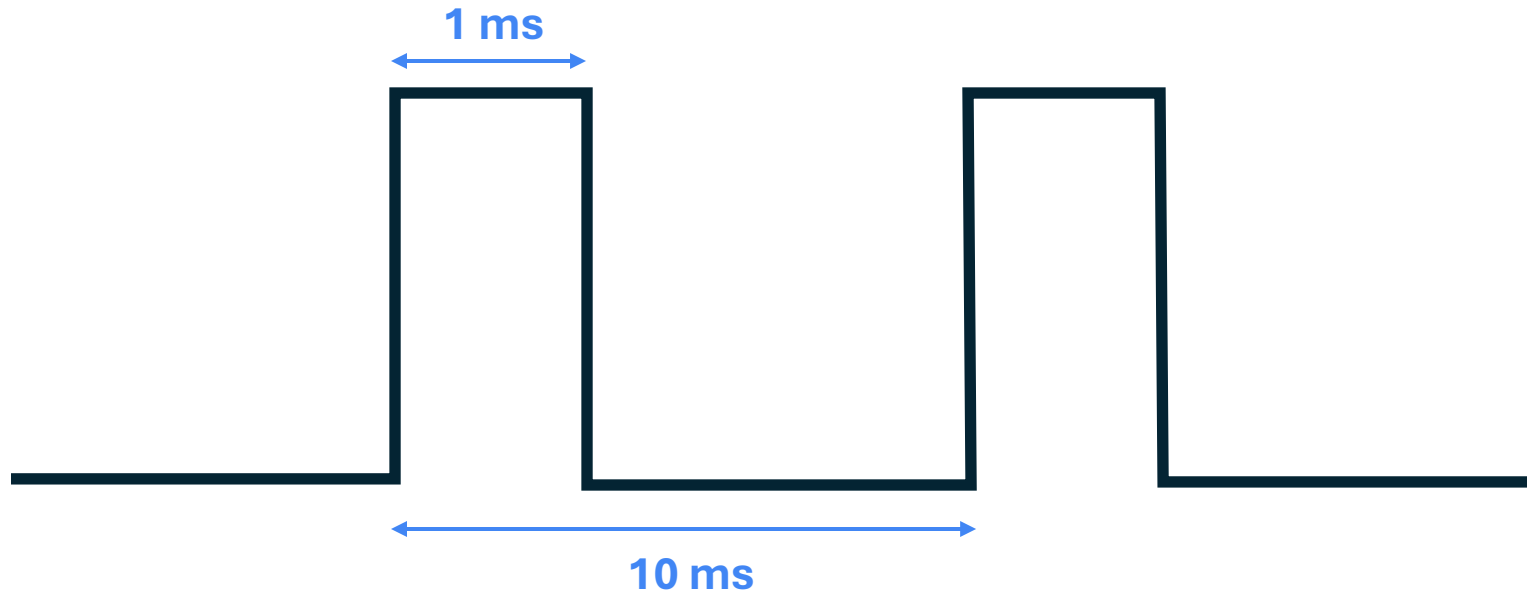
PWM Timer Setup

Given the following assumptions, find the timer's PSC, ARR, CCR to achieve a PWM output with a duty cycle of 10% and a period of 10ms

- A 16 MHz clock is selected to drive the timer
- The timer is setup as a repeatedly up-counting and down-counting, similar to the figure shown below
- PWM Mode 2 is selected. In this mode, the timer output is high if the counter is larger than or equal to CCR. Otherwise, the output is low.



Refresher



$$f_{clock_out} = \frac{f_{clock_in}}{PSC + 1}$$

Group Exercise

PWM Timer Setup

Given the following assumptions, find the timer's PSC, ARR, CCR to achieve a PWM output with a duty cycle of 10% and a period of 10ms

- A 16 MHz clock is selected to drive the timer
- The timer is setup as a repeatedly up-counting and down-counting, similar to the figure shown below
- PWM Mode 2 is selected. In this mode, the timer output is high if the counter is larger than or equal to CCR. Otherwise, the output is low.

$$\text{Timer Clock Frequency} = \frac{\text{Input Clock Frequency}}{1 + \text{PSC}}$$

$$\text{PWM Period} = 2 \times \text{ARR} \times \text{Timer Clock Period (center aligned mode)}$$

$$\text{PWM Duty Cycle} = \frac{1 - \text{CCR}}{\text{ARR}}$$

Group Exercise

Given an input clock of 100MHz what is the maximum timeout period for:

- 16 Bit Timer
- 32 Bit Timer
- 64 Bit Timer

Virtual Timers

Virtual Timers

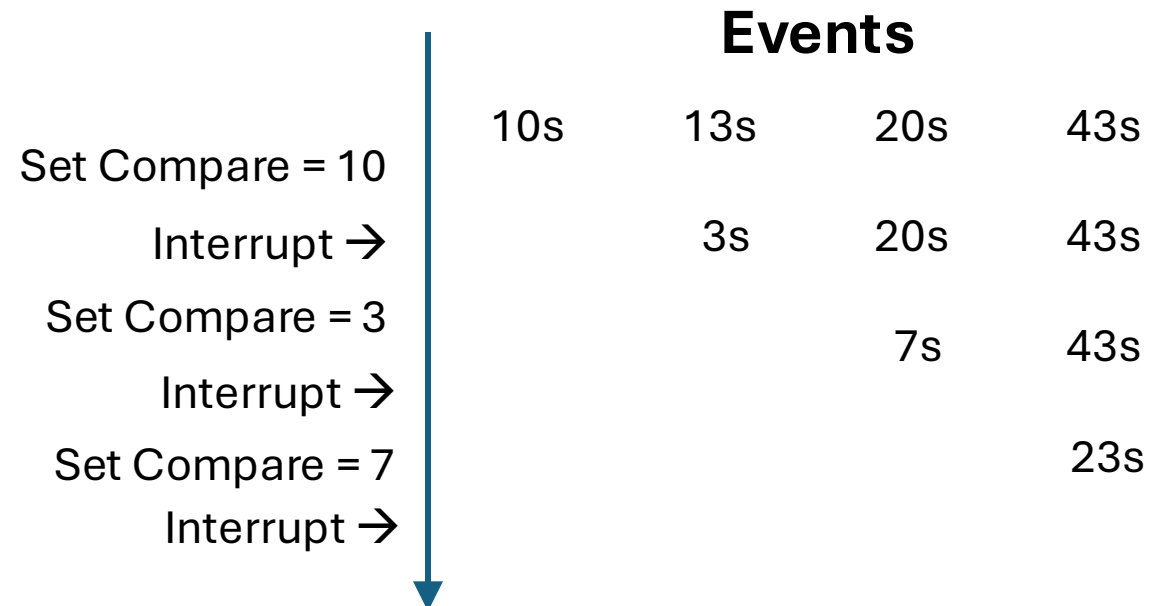
You may find that you need more timers!



How can we use software to deal with this?

Simple Idea

- Let's say we have several events that we want to generate
- Create a list and set the timer to go off for the *first* one
- Do that first task, change the timer to interrupt for the next task



Recap

- Oscillators
- Clock Distribution and Prescalers
- Timers
 - Capture and Compare
- PWM Generation and Timer Modes
- Virtual Timers